

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО”**

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

До захисту допущено:

Завідувач кафедри

_____ Сергій СТИРЕНКО
(підпис)

“__” _____ 2025 р.

Дипломний проєкт

на здобуття ступеня бакалавра

**за освітньо-професійною програмою “Інженерія програмного
забезпечення комп’ютерних систем”**

спеціальності 121 “Інженерія програмного забезпечення”

на тему: Система порівняльного тестування ефективності підходів DOTS та
ООР в середовищі Unity

Виконав : студент 4 курсу, групи ІМ-11
(шифр групи)

_____ Олійник Віктор Володимирович
(прізвище, ім’я, по батькові) (підпис)

Керівник _____ асистент Череватенко О. В.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Консультант (нормоконтроль) асистент Пономаренко А.М.
(назва розділу) (посада, вчене звання, науковий ступінь, прізвище та ініціали) (підпис)

Рецензент _____
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Засвідчую, що у цьому дипломному
проєкті немає запозичень з праць інших
авторів без відповідних посилань.

Студент _____
(підпис)

Київ – 2025 р.

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО”**

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Рівень вищої освіти – перший (бакалавр)

Освітньо-професійна програма

“Інженерія програмного забезпечення комп’ютерних систем”

спеціальності 121 “Інженерія програмного забезпечення”

ЗАТВЕРДЖУЮ

Завідувач кафедри

Сергій СТИРЕНКО

(підпис)

“__” _____ 2025 р.

ЗАВДАННЯ

на бакалаврський дипломний проєкт студента

Олійника Віктора Володимировича

1. Тема проєкту Система порівняльного тестування ефективності підходів DOTS та OOP в середовищі Unity

керівник проєкту

(прізвище, ім’я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від 11 травня 2025 року №1139-с

2. Термін здачі студентом закінченого проєкту

3. Вихідні дані до проєкту технічна документація, теоретичні дані.

4. Зміст розрахунково-пояснювальної записки (перелік питань, які розробляються)

Розділ 1. Огляд еволюційних алгоритмів для вирішення задачі глобальної оптимізації.

Розділ 2. Огляд алгоритмів та технологій для розробки системи.

Розділ 3. Деталі розробки системи.

Розділ 4. Дослідження та аналіз розробленої системи.

5. Перелік графічного матеріалу (з точним позначенням обов'язкових креслень) структурна схема системи, функціональна схема (діаграма класів), алгоритм дій програмного забезпечення.

6. Консультанта проєкту, з вказівкою розділів проєкту, які до них вносяться

Розділ	Консультант	Підпис, дата	
		Завдання видав	Завдання прийняв
Нормоконтроль	Пономаренко А.М.		

7. Дата видачі завдання

Календарний план

№ п/п	Найменування етапів дипломного проєкту	Терміни виконання етапів проєкту	Примітки
1.	<i>Затвердження теми проєкту</i>	30.01.2025-20.02.2025	
2.	<i>Вивчення та аналіз завдання</i>	21.02.2025-01.03.2025	
3.	<i>Розробка архітектури та загальної структури системи</i>	02.03.2025-12.03.2025	
4.	<i>Розробка структур окремих підсистем</i>	13.03.2025-20.03.2025	
5.	<i>Програмна реалізація системи</i>	21.03.2025-31.04.2025	
6.	<i>Оформлення пояснювальної записки</i>	01.05.2025-02.06.2025	
7.	<i>Захист програмного продукту</i>	06.06.2025	
8.	<i>Передзахист</i>	08.06.2025	
9.	<i>Захист</i>	17.06.2025	

Студент-дипломник _____ Віктор ОЛІЙНИК
(підпис)

Керівник проєкту _____ Олексій ЧЕРЕВАТЕНКО
(підпис)

АНОТАЦІЯ

Дане дослідження порівнює продуктивність двох підходів до розробки в Unity: традиційного об'єктно-орієнтованого MonoBehaviour та дата-орієнтованого DOTS. Шляхом імплементації алгоритму пошуку шляху A* з використанням обох методів, дослідження демонструє, що DOTS забезпечує вищу продуктивність та масштабованість для проєктів з великою кількістю об'єктів, тоді як MonoBehaviour пропонує кращу зручність для прототипування та проєктів меншого масштабу. Розроблена система тестування надає розробникам інструментарій для прийняття обґрунтованих архітектурних рішень на основі конкретних вимог їхніх проєктів.

Ключові слова: Unity, DOTS, об'єктно-орієнтоване програмування, алгоритм A*, оптимізація продуктивності, порівняльне тестування.

ANNOTATION

This research compares the performance of two Unity development approaches: traditional object-oriented MonoBehaviour and data-oriented DOTS. Through implementation of the A* pathfinding algorithm using both methods, the study demonstrates that DOTS provides superior performance and scalability for projects with numerous objects, while MonoBehaviour offers better convenience for prototyping and smaller-scale projects. The testing system developed provides developers with a framework to make informed architectural decisions based on their specific project requirements.

Keywords: Unity, DOTS, object-oriented programming, A* algorithm, performance optimization, comparative testing.

[illegible]

**ТЕХНІЧНЕ ЗАВДАННЯ
ДО ДИПЛОМНОГО ПРОЄКТУ**

на тему: «Система порівняльного тестування ефективності підходів DOTS
та ООР в середовищі Unity»

Київ – 2025

ЗМІСТ

НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ.....	2.
ПІДСТАВИ ДЛЯ РОЗРОБКИ.....	2.
МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ.....	2.
ДЖЕРЕЛА РОЗРОБКИ.....	3.
ТЕХНІЧНІ ВИМОГИ	3.
Вимоги до розробленого продукту	3
Вимоги до програмного забезпечення	3
Вимоги до апаратної частини	4
ЕТАПИ РОЗРОБКИ.....	4

					ІАЛЦ.467200.002 ТЗ					
Змн.	Арк.	№ докум.	Підпис	Дата	Система порівняльного тестування ефективності підходів DOTS та ООР в середовищі Unity Технічне завдання			Літ.	Арк.	Акрушів
Розроб.		Олійник В.В.								
Перевір.		Череватенко О.В.							1	4
								КПІ ім. Ігоря Сікорського, ФІОТ, ІМ-11		
Н. Контр.		Пономаренко А.М.								
Затверд.										

1 НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ

Дане технічне завдання поширюється на розробку системи порівняльного аналізу продуктивності підходів MonoBehavior OOP та Data-Oriented Technology Stack (DOTS) в середовищі Unity, включаючи імплементацію алгоритму A* та оцінку ефективності відповідних фізичних рушіїв.

Областю застосування цієї системи є розробка високопродуктивних застосунків на базі Unity, в тому числі ігор, симуляцій, візуалізацій та інших інтерактивних програм, які потребують оптимальної продуктивності при роботі з великою кількістю об'єктів, складними обчисленнями та фізичними симуляціями.

2 ПІДСТАВИ ДЛЯ РОЗРОБКИ

Підставою для розробки даної системи є завдання для виконання роботи кваліфікаційно-освітнього рівня «бакалавр інженерії програмного забезпечення», який був затверджений факультетом "Інформатики та обчислювальної техніки" кафедрою обчислювальної техніки Національного технічного Університету України «Київський Політехнічний інститут ім. Ігоря Сікорського».

3 МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ

Метою та призначенням даної роботи є дослідження та порівняльний аналіз продуктивності підходів MonoBehavior OOP та DOTS в середовищі Unity, імплементація алгоритму пошуку шляху A* із використанням обох підходів. Результати дослідження дозволять розробникам обирати оптимальний підхід в залежності від конкретних вимог проекту.

					ІАЛЦ.467200.002 ТЗ	Арк.
						2
Змн.	Арк.	№ докум.	Підпис	Дата		

4 ДЖЕРЕЛА РОЗРОБКИ

Джерелом розробки даного дипломного проекту є офіційна документація Unity, публікації та статті в мережі Інтернет на дану тему, науково-технічна література, матеріали конференцій Unity та відкриті репозиторії з імплементаціями алгоритмів та прикладами використання обох технологій.

5 ТЕХНІЧНІ ВИМОГИ

5.1. Вимоги до розробленого продукту

Розроблена система має виконувати такі вимоги:

- Реалізація алгоритму пошуку шляху A^* з використанням підходу MonoBehavior OOP.
- Реалізація алгоритму пошуку шляху A^* з використанням підходу DOTS (ECS, Job System, Burst Compiler).
- Створення тестових сценаріїв для порівняння продуктивності обох підходів з різними розмірами сітки та кількістю об'єктів.
- Розробка системи збору та візуалізації метрик продуктивності для обох підходів (час виконання, використання пам'яті, навантаження на CPU/GPU).
- Простий і інтуїтивно-зрозумілий інтерфейс для запуску тестів та перегляду результатів.
- Можливість налаштування параметрів тестування (розмір сітки, кількість агентів, складність перешкод).
- Надання вичерпної та зрозумілої документації для розробленого продукту з рекомендаціями щодо використання обох підходів.

5.2. Вимоги до програмного забезпечення

					ІАЛЦ.467200.002 ТЗ	Арк.
						3
Змн.	Арк.	№ докум.	Підпис	Дата		

- ОС Windows, Mac чи Linux.
- Unity 6.1 LTS або новіша версія.
- Пакети Unity: Entities, Jobs, Burst, Unity.Mathematics.
- Середовище розробки: Visual Studio 2019 або новіше, Visual Studio Code, JetBrains Rider.

5.3. Вимоги до апаратної частини

- ЦП не менше ніж Intel® Core i5 6-го покоління або еквівалент AMD.
- Відеокарта з підтримкою DirectX 11 або OpenGL 4.5.
- ROM не менше ніж 100 ГБ.
- RAM не менше ніж 8 ГБ.

6 ЕТАПИ РОЗРОБКИ

Назва етапів виконання	Термін виконання
Затвердження теми роботи	10.03.2025-15.03.2025
Вивчення та аналіз завдання	15.03.2025-15.04.2025
Розробка архітектури та загальної структури системи	15.04.2025-25.04.2025
Розробка структур окремих частин системи	25.04.2025-10.05.2025
Програмна реалізація системи	10.05.2025-21.05.2025
Виправлення помилок	21.03.2025-31.04.2025
Оформлення пояснювальної записки	01.05.2025-02.06.2025

					ІАЛЦ.467200.002 ТЗ	Арк.
						4
Змн.	Арк.	№ докум.	Підпис	Дата		

**ПОЯСНЮВАЛЬНА ЗАПИСКА
ДО ДИПЛОМНОГО ПРОЄКТУ**

на тему: «Система порівняльного тестування ефективності підходів DOT та
ООР в середовищі Unity»

ЗМІСТ

ВСТУП.....	7
РОЗДІЛ 1: ОГЛЯД ТЕХНОЛОГІЙ UNITY: MONOBEHAVIOUR ТА DOTS	9
1.1. Еволюція архітектури обчислювальних систем та її вплив на парадигми програмування.....	9
1.2. Загальний огляд архітектури Unity	10
1.3. Класичний підхід MonoBehaviour та об'єктно-орієнтоване програмування в Unity	12
1.3.1. Життєвий цикл MonoBehaviour	13
1.3.2. Особливості управління пам'яттю.....	13
1.3.3. Взаємодія між об'єктами	14
1.3.4. Обмеження підходу MonoBehaviour	14
1.4. Data-Oriented Technology Stack (DOTS) як відповідь на сучасні виклики.....	15
1.4.1. Передумови створення DOTS.....	15
1.4.2. Філософія дата-орієнтованого програмування	15
1.4.3. Переваги дата-орієнтованого підходу	16
1.5. Компоненти DOTS: Entity Component System (ECS), Job System, Burst Compiler.....	17
1.5.1. Entity Component System (ECS)	17
1.5.2. Job System	18
1.5.3. Burst Compiler.....	18
1.5.4. Взаємодія компонентів DOTS	19

					ІАЛЦ.467200.002 ТЗ			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Олійник В.В.						
Перевір.		Череватенко О.В.						
Н. Контр.		Пономаренко А.М.						
Затверд.								
					Лім.			Арк.
								Акрушів
								1
								79
					Система порівняльного тестування ефективності підходів DOTS та ООР в середовищі Unity			
					Пояснювальна записка			
					КПІ ім. Ігоря Сікорського, ФІОТ, ІМ-11			

1.6. Різниця в підходах до розробки: OOP vs Data-Oriented	19
1.6.1. Концептуальні відмінності.....	20
1.6.2. Відмінності в структуруванні коду	20
1.6.3. Відмінності в організації даних	21
1.6.4. Підхід до розв'язання проблем	22
1.6.5. Переваги та недоліки обох підходів.....	23
1.7. Особливості імплементації алгоритму A* та фізичних систем в Unity	24
1.7.1. Алгоритм A*	24
1.7.2. Реалізація A* з використанням MonoBehaviour:	24
1.7.3. Реалізація A* з використанням DOTS:	24
1.8. Порівняльний аналіз підходів.....	25
Ключові висновки	26
ВИСНОВОК ДО РОЗДІЛУ 1	27
РОЗДІЛ 2: АНАЛІЗ ТА ПОРІВНЯННЯ MONOBHAVIOUR ТА DOTS.....	29
2.1. Детальний аналіз архітектури MonoBehaviour	29
2.1.1. Внутрішня структура та організація пам'яті	29
2.1.2. Система подій та повідомлень.....	29
2.1.3. Продуктивність та обмеження.....	30
2.2. Комплексний аналіз архітектури DOTS	30
2.2.1. Entity Component System (ECS)	31
2.2.2. Job System	32
2.2.3. Burst Compiler	33
2.2.4. Взаємодія компонентів DOTS	34

2.3. Аналіз продуктивності MonoBehaviour у контексті багатоядерних систем	34
2.3.1. Основні обмеження MonoBehaviour для багатоядерних систем	34
2.3.2. Технічні проблеми для паралельних обчислень	35
2.3.3. Проблеми з кешем процесора	35
2.3.4. Підходи до оптимізації в рамках MonoBehaviour	36
2.4. Аналіз оптимізацій DOTS для сучасних CPU архітектур	36
2.4.1. Локальність даних та кеш-оптимізації	36
2.4.2. Векторизація та SIMD-оптимізації	37
2.4.3. Механізми паралелізації	38
2.4.4. Неблокуючі операції та асинхронність	39
2.4.5. Оптимізації для різних апаратних платформ	39
2.5. Методологія порівняння продуктивності для алгоритму A*	40
2.5.1. Принципи тестування	40
2.6.2. Сценарії тестування	41
2.6.3. Метрики продуктивності	41
2.6.4. Інструменти профілювання	42
2.6.5. Методика проведення тестів	42
2.6.6. Контроль чинників, що впливають на результати	43
2.7. Критерії вибору архітектури для різних типів проектів	43
2.7.1. Технічні критерії	44
2.7.2. Проектні критерії	45
2.7.3. Технологічні критерії	46
2.7.4. Матриця рішень для різних типів проектів	47
2.7.5. Підхід до прийняття рішення	48

2.8. Метрики та методи вимірювання	Error! Bookmark not defined.
2.8.1. Система збору метрик	Error! Bookmark not defined.
2.8.2. Інструменти збору даних.....	Error! Bookmark not defined.
2.8.3. Статистичний аналіз результатів.....	Error! Bookmark not defined.
ВИСНОВОК ДО РОЗДІЛУ 2	49
РОЗДІЛ 3: РОЗРОБКА СИСТЕМИ ПОРІВНЯЛЬНОГО ТЕСТУВАННЯ	50
3.1. Архітектура системи тестування.....	50
3.1.1. Загальна структура проєкту та принципи організації	50
3.1.2. Спільні компоненти	51
3.1.3. Ізоляція реалізацій для справедливого порівняння	52
3.2. Реалізація A* алгоритму в обох підходах	53
3.2.1. Специфіка DOTS імплементації.....	53
3.2.2. Специфіка MonoBehaviour імплементації	55
3.2.3. Забезпечення ідентичності алгоритму	56
3.3. Система збору метрик та мануального тестування	57
3.3.1. Unity Profiler налаштування для build версії.....	57
3.3.2. Збір метрик через DataCollector.....	59
3.3.3. Мануальна процедура тестування.....	60
3.4. Обробка та візуалізація даних тестування.....	63
3.4.1. Конвертація JSON в CSV формат.....	63
3.4.3. Аналіз результатів через візуалізацію	64
3.5. Користувачський інтерфейс системи	65
3.5.1. UI компоненти для налаштування.....	65
3.5.2. Перемикання між реалізаціями	66
3.5.3. Відображення поточних результатів.....	66

3.6. Технічні особливості та обмеження.....	67
3.6.1. Обмеження DOTS реалізації.....	67
3.6.2. Результати проведених тестів.....	68
ВИСНОВОК ДО РОЗДІЛУ 3	74
ВИСНОВКИ.....	75
СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ	78

					ІАЛЦ.467200.003 ПЗ	Арк.
						5
Змн.	Арк.	№ докум.	Підпис	Дата		

ПЕРЕЛІК СКОРОЧЕНЬ

DOTS (Data-Oriented Technology Stack) — Стек технологій орієнтованих на дані

ECS (Entity Component System) — Система сутностей та компонентів

SoA (Structure of Arrays) — Структура масивів

AoS (Array of Structures) — Масив структур

SIMD (Single Instruction, Multiple Data) — Одна інструкція, багато даних

PhysX — Фізичний рушій для MonoBehaviour

IL2CPP — Компіляція проміжної мови в C++

CPU (Central Processing Unit) — Центральний процесор

GPU (Graphics Processing Unit) — Графічний процесор

A* — Алгоритм пошуку шляху

FPS (Frames Per Second) — Кадрів за секунду

OOP (Object-Oriented Programming) — Об'єктно-орієнтоване програмування

DOP (Data-Oriented Programming) — Програмування орієнтоване на дані

					ІАЛЦ.467200.003 ПЗ	Арк.
						6
Змн.	Арк.	№ докум.	Підпис	Дата		

ВСТУП

У сучасному світі розробки програмного забезпечення для інтерактивних застосунків та ігрових систем оптимізація продуктивності є одним із ключових завдань, що постають перед розробниками. Саме від ефективності використання обчислювальних ресурсів залежить плавність відображення контенту, швидкість реакції на дії користувача та загальна якість кінцевого продукту. Ігровий рушій Unity, будучи одним із найпопулярніших інструментів для створення інтерактивних застосунків, пропонує розробникам два фундаментально відмінні підходи до організації програмного коду: традиційний об'єктно-орієнтований підхід MonoBehaviour та сучасний дата-орієнтований підхід DOTS (Data-Oriented Technology Stack).

Актуальність дослідження порівняння продуктивності MonoBehaviour та DOTS обумовлена стрімким розвитком сучасних технологій та зростаючими вимогами до продуктивності програмних систем. Розробники застосунків на базі Unity постійно стикаються з необхідністю вибору між перевіреним часом об'єктно-орієнтованим підходом та інноваційною парадигмою дата-орієнтованого програмування. Особливо гостро питання вибору архітектури постає при розробці ресурсомістких компонентів, таких як складні алгоритми пошуку шляху (наприклад, A^*) та фізичні симуляції, де будь-яке зниження продуктивності може призвести до незадовільного користувацького досвіду.

Метою даної роботи є проведення комплексного порівняльного аналізу продуктивності підходів MonoBehaviour OOP та DOTS в середовищі Unity на прикладі алгоритму пошуку шляху A^* . Для досягнення поставленої мети необхідно розробити систему, яка дозволить реалізувати алгоритм A^* та фізичні симуляції з використанням обох підходів, провести тестування в різних

					ІАЛЦ.467200.003 ПЗ	Арк.
						7
Змн.	Арк.	№ докум.	Підпис	Дата		

умовах навантаження та зібрати метрики продуктивності для подальшого аналізу.

У рамках роботи буде здійснено імплементацію алгоритму A* з використанням класичного підходу MonoBehaviour та сучасної архітектури DOTS, що дозволить встановити переваги та недоліки кожного з методів в конкретному застосуванні. Крім того, буде проведено порівняння ефективності фізичних рушіїв, що відповідають кожному з підходів: стандартного PhysX для MonoBehaviour та Unity Physics для DOTS.

Результати дослідження матимуть практичну цінність для розробників, які прагнуть обрати оптимальний підхід до архітектури своїх проектів залежно від конкретних вимог та обмежень. Дана робота надасть об'єктивні дані та рекомендації, які допоможуть приймати обґрунтовані рішення щодо використання MonoBehaviour або DOTS в різних сценаріях розробки застосунків на базі Unity

					ІАЛЦ.467200.003 ПЗ	Арк.
						8
Змн.	Арк.	№ докум.	Підпис	Дата		

РОЗДІЛ 1: ОГЛЯД ТЕХНОЛОГІЙ UNITY: MONOBHAVIOUR TA DOTS

1.1. Еволюція архітектури обчислювальних систем та її вплив на парадигми програмування

Розвиток обчислювальних систем пройшов довгий шлях від однопроцесорних архітектур до сучасних багатоядерних рішень. До початку 2000-х років основним способом підвищення продуктивності комп'ютерів було збільшення тактової частоти процесорів. Цей підхід, проте, зіткнувся з фундаментальними фізичними обмеженнями, пов'язаними з тепловиділенням та енергоспоживанням.

Ця проблема, відома як "стіна тепловиділення" (power wall), змусила виробників процесорів змінити стратегію розвитку. Замість подальшого підвищення частоти, вони почали збільшувати кількість ядер в одному чіпі. Так відбувся перехід до багатоядерних архітектур, що спостерігається і в сучасних мобільних пристроях, і в потужних серверних рішеннях.

Цей перехід суттєво вплинув на парадигми програмування. Традиційний послідовний підхід до виконання коду, характерний для об'єктно-орієнтованого програмування (ООП), виявився неефективним у контексті багатоядерних систем. ООП, з його фокусом на інкапсуляції даних та поведінки в об'єктах, створює структури, що складно розпаралелюються без істотних модифікацій коду.

					ІАЛЦ.467200.003 ПЗ	Арк.
						9
Змн.	Арк.	№ докум.	Підпис	Дата		

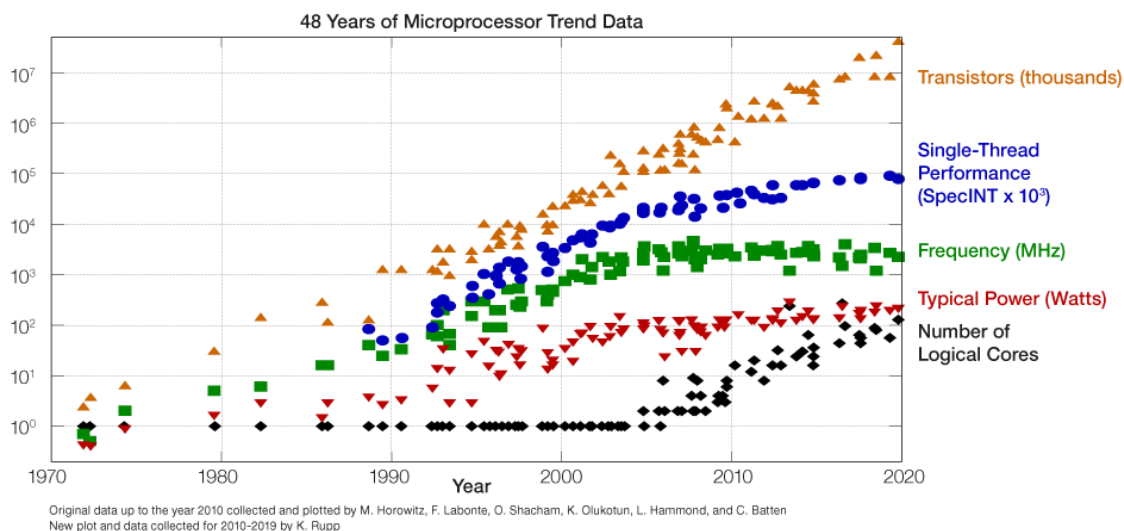


Рис. 1.1. «Граф тренду розвитку мікропроцесорів»

Проблеми, з якими стикається ООП на багатоядерних системах:

1. **Розподіл даних у пам'яті:** Об'єкти в ООП часто розкидані в різних областях пам'яті, що призводить до неефективного використання кешу процесора.
2. **Складність паралелізації:** Взаємозалежності між об'єктами ускладнюють паралельне виконання операцій над ними.
3. **Витрати на віртуальні виклики:** Поліморфізм та віртуальні методи створюють додаткові накладні витрати на виконання.

Ці обмеження призвели до пошуку нових підходів, які б краще відповідали архітектурі сучасних обчислювальних систем. Одним з таких підходів стало дата-орієнтоване програмування (Data-Oriented Programming), яке фокусується на ефективній організації даних для оптимального використання можливостей апаратного забезпечення, особливо кеш-пам'яті та векторних інструкцій процесора.

1.2. Загальний огляд архітектури Unity

Unity — це потужний кросплатформений рушій для розробки ігор та інтерактивних додатків. Архітектура Unity базується на компонентно-орієнтованому підході, який дозволяє створювати складні системи з модульних будівельних блоків.

					ІАЛЦ.467200.003 ПЗ	Арк.
						10
Змн.	Арк.	№ докум.	Підпис	Дата		

Основні компоненти архітектури Unity:

1. **GameObject:** Базовий контейнер для всіх компонентів у сцені Unity. Самі по собі GameObject не мають функціональності, але служать "вузлами" для прикріплення різних компонентів.
2. **Component:** Модулі, що додають функціональність до GameObject. Кожен компонент відповідає за конкретний аспект функціональності (трансформація, рендеринг, фізика, аудіо, тощо).
3. **Scene:** Контейнер для організації об'єктів, представляє окремий рівень, меню або простір взаємодії.
4. **Asset:** Ресурси, які використовуються в проєкті (текстури, моделі, звуки, скрипти).
5. **Prefab:** Шаблони для створення екземплярів GameObject з попередньо налаштованими компонентами.

Життєвий цикл Unity-додатку складається з таких основних етапів:

- **Ініціалізація:** Завантаження ресурсів, створення об'єктів та компонентів.
- **Цикл оновлення:** Обробка вводу, оновлення стану об'єктів, фізичних симуляцій, рендеринг.
- **Завершення роботи:** Звільнення ресурсів, збереження даних.

					ІАЛЦ.467200.003 ПЗ	Арк.
						11
Змн.	Арк.	№ докум.	Підпис	Дата		

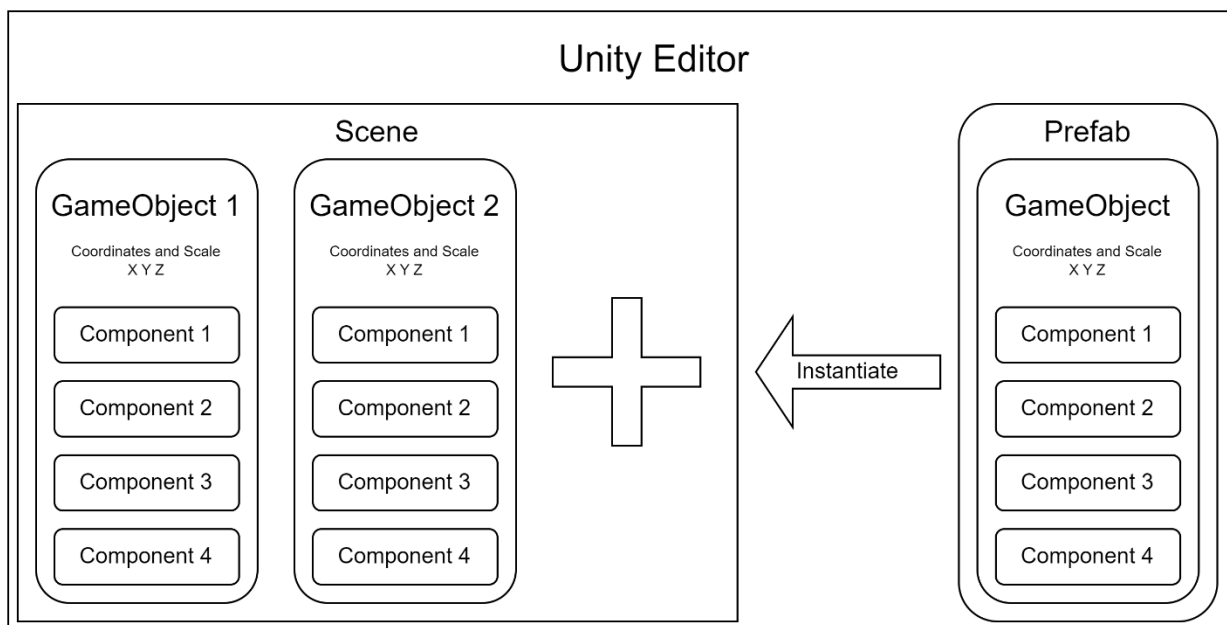


Рис. 1.2. . «Класична архітектура проекту при роботі в UnityEditor»

Взаємодія між компонентами в Unity реалізується через посилання, події та повідомлення. Компоненти можуть отримувати доступ до інших компонентів, прикріплених до того ж GameObject або до інших об'єктів у сцені.

Цей компонентно-орієнтований підхід дозволяє створювати гнучкі та модульні системи, але традиційна реалізація через MonoBehaviour має певні обмеження з точки зору продуктивності, особливо при роботі з великою кількістю об'єктів.

1.3. Класичний підхід MonoBehaviour та об'єктно-орієнтоване програмування в Unity

MonoBehaviour — це базовий клас для всіх скриптів у традиційному підході Unity, що реалізує об'єктно-орієнтовану парадигму. Скрипти, що успадковуються від MonoBehaviour, можуть бути прикріплені до GameObject для визначення їхньої поведінки.

1.3.1. Життєвий цикл MonoBehaviour

Скрипти MonoBehaviour мають доступ до ряду методів життєвого циклу, які викликаються рушієм у визначені моменти:

- **Awake():** Викликається один раз при завантаженні об'єкта, навіть якщо скрипт неактивний.
- **Start():** Викликається перед першим кадром, якщо скрипт активний.
- **Update():** Викликається кожен кадр.
- **FixedUpdate():** Викликається з фіксованою частотою, використовується для фізичних обчислень.
- **LateUpdate():** Викликається після всіх Update(), використовується для остаточних модифікацій.
- **OnEnable()/OnDisable():** Викликаються при активації/деактивації компонента.
- **OnDestroy():** Викликається при знищенні об'єкта.

Цей життєвий цикл є простим для розуміння і використання, але створює певні проблеми для продуктивності:

1. **Послідовне виконання** - Методи життєвого циклу виконуються послідовно для кожного об'єкта, що обмежує можливості паралелізації.
2. **Віртуальні виклики** - Кожен виклик методу життєвого циклу є віртуальним, що створює додаткові накладні витрати.
3. **Пропуски методів** - Якщо метод життєвого циклу не реалізований в скрипті, Unity все одно витрачає ресурси на його виклик і перевірку.

1.3.2. Особливості управління пам'яттю

Unity використовує автоматичне керування пам'яттю через збирач сміття (garbage collector) .NET. Це спрощує розробку, але може призводити до проблем з продуктивністю:

1. **Фрагментація пам'яті:** Об'єкти створюються та видаляються в різних місцях пам'яті, що призводить до її фрагментації.

					ІАЛЦ.467200.003 ПЗ	Арк.
						13
Змн.	Арк.	№ докум.	Підпис	Дата		

2. **Зупинки збирача сміття:** Періодичні паузи для очищення пам'яті можуть викликати помітні затримки в роботі додатку.
3. **Розподілена структура даних:** Компоненти одного типу можуть бути розкидані в пам'яті, що погіршує ефективність використання кешу процесора.

1.3.3. Взаємодія між об'єктами

В підході MonoBehaviour взаємодія між об'єктами здійснюється кількома способами:

1. **Прямі посилання:** Отримання посилань на інші компоненти через методи GetComponent, FindObjectOfType тощо.
2. **Шаблон спостерігача:** Використання подій та делегатів для слабкого зв'язування об'єктів.
3. **Системи повідомлень:** SendMessage та BroadcastMessage для відправлення повідомлень між об'єктами.

1.3.4. Обмеження підходу MonoBehaviour

Хоча MonoBehaviour надає інтуїтивно зрозумілий інтерфейс для розробників, цей підхід має кілька суттєвих обмежень:

1. **Неоптимальне використання кешу процесора:** Дані різних типів розміщуються поруч в об'єкті, але обробка часто потребує лише частини цих даних, що призводить до неефективного використання кешу.
2. **Обмежені можливості для паралелізації:** Код виконується переважно в одному потоці, що не дозволяє повністю використати потенціал багатоядерних процесорів.
3. **Накладні витрати на віртуальні виклики:** Поліморфізм та віртуальні методи створюють додаткові витрати на виконання.
4. **Проблеми з масштабованістю:** При збільшенні кількості об'єктів продуктивність значно падає.

					ІАЛЦ.467200.003 ПЗ	Арк.
						14
Змн.	Арк.	№ докум.	Підпис	Дата		

Ці обмеження особливо помітні в сценаріях з великою кількістю об'єктів, складними фізичними симуляціями або на платформах з обмеженими ресурсами, що призвело до розробки альтернативного підходу — Data-Oriented Technology Stack (DOTS).

1.4. Data-Oriented Technology Stack (DOTS) як відповідь на сучасні виклики

Data-Oriented Technology Stack (DOTS) — це набір технологій, розроблених Unity для подолання обмежень традиційного об'єктно-орієнтованого підходу та кращого використання можливостей сучасного апаратного забезпечення, особливо багатоядерних процесорів.

1.4.1. Передумови створення DOTS

Розробка DOTS була зумовлена кількома ключовими факторами:

1. **Еволюція процесорних архітектур:** Перехід від нарощування частоти до збільшення кількості ядер вимагав нових підходів до написання коду.
2. **Обмеження традиційного підходу:** MonoBehaviour показував недостатню продуктивність у сценаріях з великою кількістю об'єктів.
3. **Потреба в кросплатформеності:** Необхідність ефективно працювати на різних пристроях, від мобільних до ігрових консолей.

1.4.2. Філософія дата-орієнтованого програмування

На відміну від об'єктно-орієнтованого підходу, який фокусується на об'єднанні даних і поведінки, дата-орієнтоване програмування зосереджується на ефективній організації даних для оптимального використання апаратного забезпечення:

1. **Фокус на організації даних:** Структури даних організовуються для максимальної ефективності обробки, а не для концептуальної чистоти.
2. **Локальність даних:** Дані, що обробляються разом, розташовуються поруч у пам'яті для кращого використання кешу.

					ІАЛЦ.467200.003 ПЗ	Арк.
						15
Змн.	Арк.	№ докум.	Підпис	Дата		

3. **Розділення даних і логіки:** Чітке розмежування між даними (стан) і логікою (системи, що обробляють ці дані).
4. **Трансформація даних:** Акцент на потоках даних та їх трансформації, а не на об'єктах і методах.

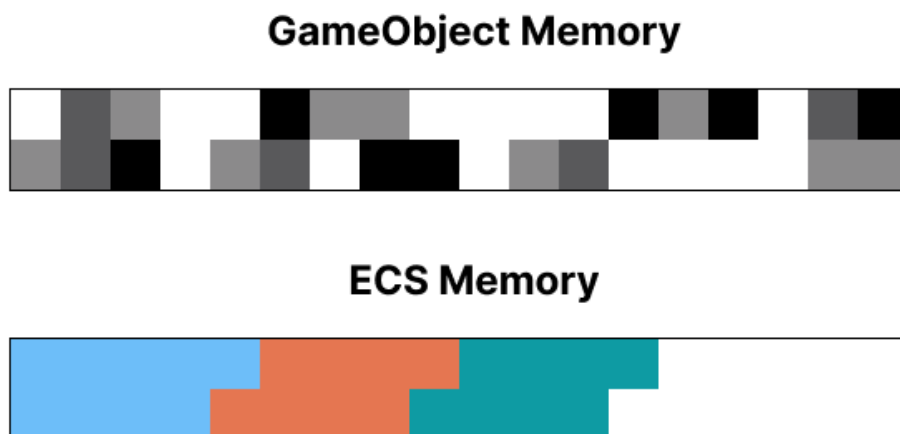


Рис. 1.3 «Розподілення пам'яті в MonoBehaviour та DOTS»

1.4.3. Переваги дата-орієнтованого підходу

Дата-орієнтований підхід, реалізований в DOTS, пропонує ряд важливих переваг:

1. **Вища продуктивність:** Ефективніше використання кешу процесора та можливість паралельного виконання.
2. **Краща масштабованість:** Можливість обробляти значно більше об'єктів без пропорційного зниження продуктивності.
3. **Детермінізм:** Передбачувана поведінка системи, що спрощує налагодження та тестування.
4. **Ефективне використання багатоядерних процесорів:** Можливість розподілу навантаження між усіма доступними ядрами.

DOTS представляє значну зміну парадигми розробки в Unity, пропонуючи потужну альтернативу традиційному підходу MonoBehaviour, особливо для проєктів, де продуктивність є критичним фактором.

1.5. Компоненти DOTS: Entity Component System (ECS), Job System, Burst Compiler

DOTS складається з трьох основних технологій, які працюють разом для забезпечення високої продуктивності та ефективного використання сучасного апаратного забезпечення:

1.5.1. Entity Component System (ECS)

ECS — це архітектурний патерн, який розділяє дані (компоненти) від логіки (системи) та ідентичності об'єктів (сутності):

1. **Entity (Сутність):** Легкий ідентифікатор, який представляє унікальний об'єкт у грі або додатку. Сутність сама по собі не містить даних або логіки, а лише служить для групування компонентів.
2. **Component (Компонент):** Чистий контейнер даних, який описує певний аспект сутності (позиція, швидкість, здоров'я тощо). Компоненти не містять логіки або методів, лише дані.
3. **System (Система):** Логіка, яка обробляє сутності з певними комбінаціями компонентів. Системи виконують трансформації даних, наприклад, оновлення позицій на основі швидкості.

Такий поділ дозволяє організувати дані оптимальним чином, групуючи компоненти одного типу разом у пам'яті, що значно покращує використання кешу процесора.

					ІАЛЦ.467200.003 ПЗ	Арк.
						17
Змн.	Арк.	№ докум.	Підпис	Дата		

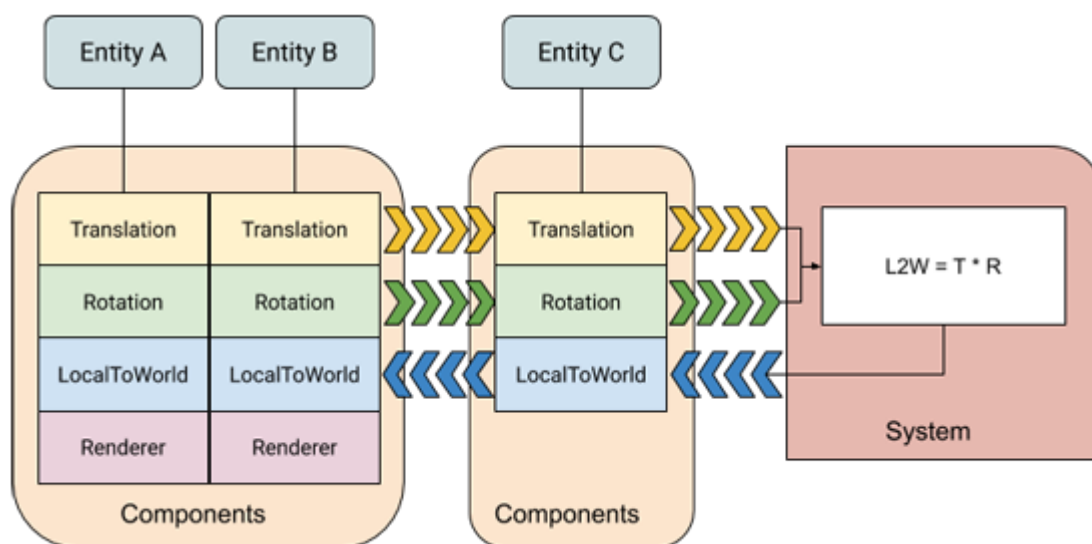


Рис. 1.4 «Архітектура Entity-Component-System»

1.5.2. Job System

Job System — це фреймворк для організації паралельних обчислень, який дозволяє ефективно розподіляти роботу між доступними ядрами процесора:

1. **Job:** Невелика одиниця роботи, яка може виконуватися паралельно з іншими завданнями.
2. **Dependency Management:** Автоматичне керування залежностями між завданнями для уникнення конфліктів доступу до даних.
3. **Worker Threads:** Пул потоків, який обробляє завдання та оптимально розподіляє навантаження.

Job System дозволяє писати багатопотоковий код без необхідності вручну керувати потоками або розв'язувати проблеми синхронізації.

1.5.3. Burst Compiler

Burst Compiler — це технологія компіляції, яка перетворює статично типізований код .NET IL у високооптимізований нативний машинний код:

1. **SIMD-оптимізації:** Автоматичне використання векторних інструкцій процесора для паралельної обробки даних.

2. **Математичні оптимізації:** Агресивна оптимізація математичних операцій.
3. **Інлайнінг функцій:** Вбудовування функцій для зменшення накладних витрат на виклики.
4. **Усунення перевірок безпеки:** Видалення непотрібних перевірок для підвищення продуктивності.

Burst Compiler особливо ефективний при роботі з математично інтенсивними операціями, такими як фізичні симуляції або обробка великих масивів даних.

1.5.4. Взаємодія компонентів DOTS

Всі три технології DOTS працюють разом, створюючи потужний інструментарій для високопродуктивної розробки:

1. **ECS** організовує дані оптимальним чином для ефективного доступу.
2. **Job System** дозволяє паралельно обробляти ці дані на всіх доступних ядрах.
3. **Burst Compiler** генерує високооптимізований нативний код для максимальної продуктивності.

Така комбінація технологій дозволяє досягти значного підвищення продуктивності в порівнянні з традиційним підходом MonoBehaviour, особливо в сценаріях з великою кількістю об'єктів або складними обчисленнями.

1.6. Різниця в підходах до розробки: OOP vs Data-Oriented

Об'єктно-орієнтоване програмування (OOP) і дата-орієнтоване програмування (Data-Oriented Programming, DOP) представляють два фундаментально різних підходи до розробки програмного забезпечення, з відмінностями як у філософії, так і в практичній реалізації.

					ІАЛЦ.467200.003 ПЗ	Арк.
						19
Змн.	Арк.	№ докум.	Підпис	Дата		

1.6.1. Концептуальні відмінності

Об'єктно-орієнтований підхід фокусується на об'єктах, що поєднують дані і поведінку, створює концептуальні моделі реального світу, структурує код через ієрархію класів, забезпечує високу інкапсуляцію даних і методів в одному об'єкті, і досягає модульності на рівні об'єктів і класів.

Дата-орієнтований підхід, натомість, концентрується на даних та їх ефективній організації, оптимізує структури для продуктивності, використовує плоскі структури даних, чітко розділяє дані і логіку з низьким рівнем інкапсуляції, і реалізує модульність на рівні компонентів і систем.

1.6.2. Відмінності в структуруванні коду

В ООР код організується навколо класів, які інкапсулюють стан і поведінку:

```
public class Player : MonoBehaviour
{
    public float health;
    public float speed;

    void Update()
    {
        // Логіка руху гравця
        transform.position += new Vector3(Input.GetAxis("Horizontal"), 0, 0) *
speed * Time.deltaTime;

        // Логіка зміни здоров'я
        if (Input.GetKeyDown(KeyCode.Space))
            TakeDamage(10);
    }

    void TakeDamage(float amount)
    {
        health -= amount;
        if (health <= 0)
            Die();
    }

    void Die()
    {
        Destroy(gameObject);
    }
}
```

Рис. 1.5. «Об'єктно-орієнтований підхід (MonoBehaviour)»

					ІАЛЦ.467200.003 ПЗ	Арк.
						20
Змн.	Арк.	№ докум.	Підпис	Дата		

У DOP код структурується навколо даних і систем, які їх обробляють:

```
// Компоненти (тільки дані)
public struct Health : IComponentData
{
    public float Value;
}

public struct Movement : IComponentData
{
    public float Speed;
}

// Системи (тільки логіка)
public class MovementSystem : SystemBase
{
    protected override void OnUpdate()
    {
        float deltaTime = Time.DeltaTime;
        float horizontalInput = Input.GetAxis("Horizontal");

        Entities
            .ForEach((ref Translation position, in Movement movement) =>
            {
                position.Value += new float3(horizontalInput, 0, 0) *
movement.Speed * deltaTime;
            })
            .Schedule();
    }
}

public class DamageSystem : SystemBase
{
    protected override void OnUpdate()
    {
        bool spacePressed = Input.GetKeyDown(KeyCode.Space);

        Entities
            .WithStructuralChanges()
            .ForEach((Entity entity, ref Health health) =>
            {
                if (spacePressed)
                    health.Value -= 10;

                if (health.Value <= 0)
                    EntityManager.DestroyEntity(entity);
            })
            .Run();
    }
}
```

Рис. 1.6. «Дата-орієнтований підхід (ECS)»

1.6.3. Відмінності в організації даних

Різниця в організації даних між OOP і DOP має критичне значення для продуктивності:

					ІАЛЦ.467200.003 ПЗ	Арк.
						21
Змн.	Арк.	№ докум.	Підпис	Дата		

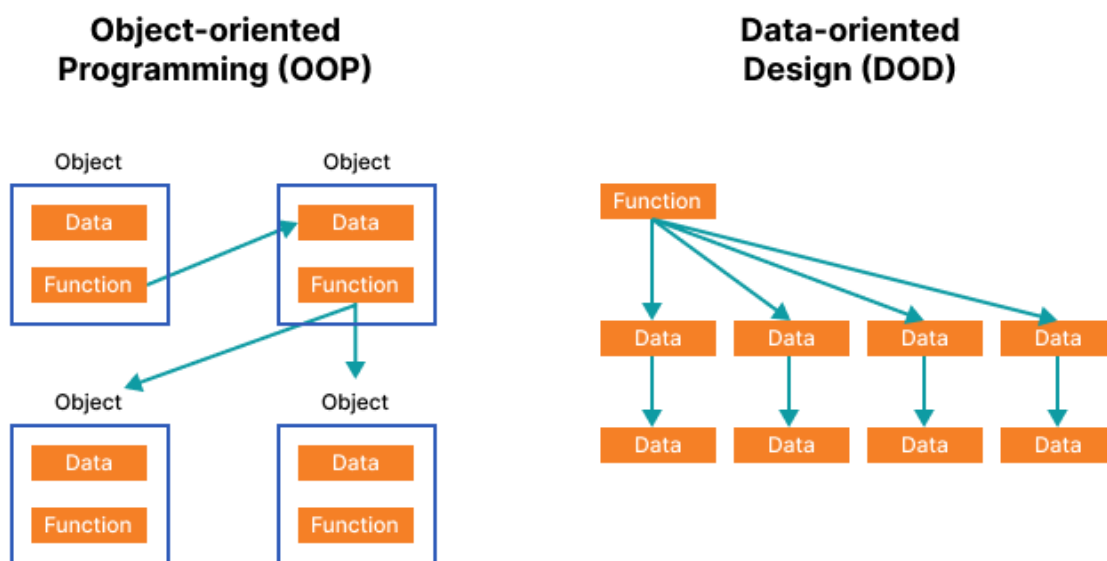


Рис. 1.7. «Порівняння організації OOP та DOD»

- **OOP:** Дані різних типів групуються в окремі об'єкти, які розкидані в пам'яті. При обробці однотипних даних (наприклад, позицій) процесор змушений завантажувати в кеш багато непотрібної інформації.
- **DOP:** Дані одного типу групуються разом у суцільних масивах (System of Arrays, SoA), що дозволяє процесору ефективно обробляти їх за один прохід, максимально використовуючи кеш.

1.6.4. Підхід до розв'язання проблем

Різні парадигми впливають на спосіб мислення розробника:

- **OOP:** Розробник думає в термінах об'єктів, їхніх властивостей і взаємодій. Фокус на моделюванні реального світу через класи та інтерфейси.
- **DOP:** Розробник думає в термінах даних, їхніх трансформацій і потоків. Фокус на ефективній організації даних для оптимальної обробки.

1.6.5. Переваги та недоліки обох підходів

Об'єктно-орієнтований підхід:

- ✓ Інтуїтивна зрозумілість: відповідає тому, як люди сприймають світ
- ✓ Гнучкість: легко додавати нові функції через успадкування та поліморфізм
- ✓ Повторне використання коду: через класи та інтерфейси
- ✗ Неоптимальне використання кешу процесора
- ✗ Обмежені можливості для паралелізації
- ✗ Проблеми з продуктивністю при великій кількості об'єктів

Дата-орієнтований підхід:

- ✓ Висока продуктивність: ефективне використання кешу і SIMD-інструкцій
- ✓ Паралелізм: легка паралельна обробка даних
- ✓ Масштабованість: можливість обробляти великі обсяги даних
- ✗ Вища складність розробки: потребує іншого мислення
- ✗ Менша інтуїтивність: не відповідає звичному сприйняттю світу
- ✗ Крутіша крива навчання: нові концепції та інструменти

Вибір між OOP і DOP залежить від конкретних вимог проєкту, особливо щодо продуктивності, масштабованості та складності. У багатьох випадках оптимальним рішенням може бути гібридний підхід, де критичні для продуктивності частини реалізуються за допомогою дата-орієнтованого підходу, а решта — за допомогою більш звичного об'єктно-орієнтованого підходу.

					ІАЛЦ.467200.003 ПЗ	Арк.
						23
Змн.	Арк.	№ докум.	Підпис	Дата		

1.7. Особливості імплементації алгоритму A* та фізичних систем в Unity

Алгоритм пошуку шляху A* і фізичні системи є одними з найресурсоеміших компонентів у розробці ігор. Вони чудово ілюструють різницю між традиційним підходом MonoBehaviour і дата-орієнтованим підходом DOTS.

1.7.1. Алгоритм A*

A* (A-star) — це алгоритм пошуку шляху, який знаходить найкоротший шлях між двома точками з урахуванням вартості переміщення і евристичної оцінки відстані до цілі.

Принцип роботи A*:

1. Обробка вузлів у порядку їх пріоритету (сума вартості шляху та евристичної оцінки).
2. Розширення шляху додаванням сусідніх вузлів до черги з пріоритетом.
3. Повторення процесу, доки не буде знайдено шлях до цілі або вичерпано всі можливі шляхи.

1.7.2. Реалізація A* з використанням MonoBehaviour:

При традиційному підході пошук шляху реалізується як окремий компонент або сервіс, який обробляє запити послідовно. Це спричиняє суттєві проблеми: кожен пошук шляху блокує інші операції в головному потоці; створюється багато тимчасових об'єктів для вузлів і списків, що призводить до неефективного використання пам'яті; а при збільшенні кількості агентів продуктивність різко падає через обмежену масштабованість.

1.7.3. Реалізація A* з використанням DOTS:

Дата-орієнтований підхід до реалізації A* забезпечує значні переваги. Job System дозволяє паралельно виконувати кілька пошуків шляху. Дані

					ІАЛЦ.467200.003 ПЗ	Арк.
						24
Змн.	Арк.	№ докум.	Підпис	Дата		

розміщуються в суцільних масивах для кращого використання кешу. Burst Compiler генерує високооптимізований нативний код для критичних обчислень. Завдяки цьому можна обробляти значно більше агентів без суттєвого падіння продуктивності.

1.8. Порівняльний аналіз підходів

Порівняння підходів до реалізації алгоритму A^* та фізичних систем в MonoBehaviour та DOTS демонструє принципові відмінності, що впливають на продуктивність:

Для алгоритму A^* :

Таблиця 1.1. Порівняння A^* алгоритму в Mono та DOTS

Аспект	MonoBehaviour	DOTS
Паралелізація	Обмежена (переважно один потік)	Висока (Job System)
Управління пам'яттю	Динамічне виділення	Контрольоване розміщення
Масштабованість	Обмежена	Висока
Оптимізація коду	Стандартний IL-код	Нативний код (Burst)

Ключові висновки

1. *Алгоритм А та фізичні системи** є ідеальними кандидатами для впровадження з використанням дата-орієнтованого підходу через їхню обчислювальну інтенсивність та можливість паралелізації.
2. **Переваги DOTS** стають особливо помітними при масштабуванні системи до великої кількості об'єктів, де традиційний підхід MonoBehaviour зазнає значного падіння продуктивності.
3. **Гібридний підхід** часто є найкращим рішенням: використання DOTS для критичних для продуктивності систем (пошук шляху, фізика, анімація) та MonoBehaviour для логіки гри та взаємодії з користувачем.
4. **Рівень складності реалізації** також варто враховувати: DOTS пропонує вищу продуктивність, але вимагає більше зусиль для розробки та налагодження.

Алгоритм А* є показовим прикладом того, як технологічний стек DOTS може докорінно покращити підхід до розробки порівняно з традиційним підходом MonoBehaviour, особливо в сценаріях з високим обчислювальним навантаженням або великою кількістю об'єктів.

					ІАЛЦ.467200.003 ПЗ	Арк.
						26
Змн.	Арк.	№ докум.	Підпис	Дата		

ВИСНОВОК ДО РОЗДІЛУ 1

У цьому розділі було розглянуто еволюцію архітектури обчислювальних систем та її вплив на парадигми програмування, особливо в контексті розробки ігор та інтерактивних додатків на Unity.

Основні висновки:

1. **Зміна парадигми процесорної архітектури** від підвищення тактової частоти до збільшення кількості ядер створила потребу в нових підходах до програмування, які б ефективно використовували можливості багатоядерних систем.
2. **Традиційний підхід MonoBehaviour**, що базується на об'єктно-орієнтованій парадигмі, має суттєві обмеження з точки зору продуктивності, особливо при роботі з великою кількістю об'єктів або складними обчисленнями.
3. **Data-Oriented Technology Stack (DOTS)** пропонує альтернативний підхід, фокусуючись на ефективній організації даних та максимальному використанні можливостей сучасного апаратного забезпечення.
4. **Три ключові компоненти DOTS** — Entity Component System (ECS), Job System та Burst Compiler — працюють разом для забезпечення високої продуктивності та масштабованості.
5. **Різниця між OOP і DOP** проявляється як у концептуальному підході до розробки, так і в практичній організації даних та коду, з відповідними перевагами та недоліками кожного підходу.
6. **Алгоритм A*** є яскравим прикладом компоненту, який значно виграє від впровадження з використанням дата-орієнтованого підходу, демонструючи потенціал для значного підвищення продуктивності.

Розуміння цих концепцій та відмінностей між підходами є важливим для прийняття обґрунтованих архітектурних рішень при розробці проєктів на Unity, зокрема при виборі між традиційним підходом MonoBehaviour та інноваційним дата-орієнтованим підходом DOTS.

					ІАЛЦ.467200.003 ПЗ	Арк.
						27
Змн.	Арк.	№ докум.	Підпис	Дата		

У наступних розділах буде детально розглянуто практичну реалізацію пошуку шляху з використанням обох підходів, а також проведено порівняльний аналіз їх продуктивності в різних сценаріях.

					ІАЛЦ.467200.003 ПЗ	Арк.
						28
Змн.	Арк.	№ докум.	Підпис	Дата		

РОЗДІЛ 2: АНАЛІЗ ТА ПОРІВНЯННЯ MONOBEHAVIOUR TA DOTS

2.1. Детальний аналіз архітектури MonoBehaviour

MonoBehaviour є основним класом для компонентів в традиційній архітектурі Unity, що представляє об'єктно-орієнтований підхід до розробки ігор. Для розуміння сильних та слабких сторін цієї архітектури, необхідно проаналізувати її внутрішню структуру та принципи функціонування.

2.1.1. Внутрішня структура та організація пам'яті

MonoBehaviour як базовий клас компонентів має кілька ключових особливостей у своїй внутрішній реалізації: **Глибока ієрархія наслідування** - MonoBehaviour наслідується від класу Behaviour, який в свою чергу наслідується від Component, потім Object. Така ієрархія забезпечує багату функціональність, але створює накладні витрати на віртуальні виклики.

Управління пам'яттю - MonoBehaviour використовує керовану пам'ять .NET з автоматичним збором сміття. Це спрощує розробку, але призводить до непередбачуваних пауз під час виконання через роботу збирача сміття.

Розподілення даних у пам'яті - Об'єкти MonoBehaviour розміщуються в керованій купі не послідовно, а в довільних місцях пам'яті. При обробці колекції об'єктів одного типу це призводить до частих промахів кешу.

2.1.2. Система подій та повідомлень

MonoBehaviour пропонує кілька способів реалізації взаємодії між об'єктами: **Прямі посилання** - Отримання посилань на інші компоненти через методи GetComponent, FindObjectOfType тощо. Це найшвидший метод комунікації, але створює сильні зв'язки між компонентами. **Повідомлення** - Методи SendMessage і BroadcastMessage дозволяють відправляти

					ІАЛЦ.467200.003 ПЗ	Арк.
						29
Змн.	Арк.	№ докум.	Підпис	Дата		

повідомлення між об'єктами. Ці методи зручні, але неефективні через використання рефлексії.

Системи подій - Використання делегатів та подій C# для комунікації. Цей підхід забезпечує слабе зв'язування, але може призводити до витоків пам'яті, якщо підписники не відписуються належним чином. **Скриптабельні об'єкти (ScriptableObjects)** - Використовуються як спільне сховище даних між різними компонентами. Це ефективний спосіб для обміну даними, але не оптимальний для частих змін.

2.1.3. Продуктивність та обмеження

Аналіз продуктивності MonoBehaviour виявляє кілька критичних обмежень: **Неефективне використання кешу процесора** - Через розрізнене розміщення об'єктів у пам'яті, процесор часто завантажує в кеш дані, які не будуть використані. **Витрати на виклики методів** - Віртуальні виклики та використання рефлексії створюють значні накладні витрати. **Обмеження паралелізму** - Більшість операцій виконується в головному потоці, обмежуючи використання багатоядерних процесорів. **Збір сміття** - Автоматичне управління пам'яттю призводить до пауз під час виконання. **Дефрагментація пам'яті** - Постійне створення та знищення об'єктів призводить до фрагментації пам'яті.

Незважаючи на ці обмеження, MonoBehaviour надає інтуїтивно зрозумілий та простий для початку роботи інтерфейс, що пояснює його популярність серед розробників, особливо тих, хто тільки починає працювати з Unity.

2.2. Комплексний аналіз архітектури DOTS

Data-Oriented Technology Stack (DOTS) представляє принципово інший підхід до архітектури ігор в Unity, фокусуючись на ефективній організації даних та максимальному використанні сучасного апаратного забезпечення.

					ІАЛЦ.467200.003 ПЗ	Арк.
						30
Змн.	Арк.	№ докум.	Підпис	Дата		

Цей підхід складається з трьох основних компонентів: Entity Component System (ECS), Job System та Burst Compiler.

2.2.1. Entity Component System (ECS)

ECS є основою архітектури DOTS, що забезпечує структурну організацію даних та логіки:

1. **Архітектура даних** - ECS використовує архітектуру "Структура масивів" (Structure of Arrays, SoA), де компоненти одного типу зберігаються разом у пам'яті, на відміну від "Масиву структур" (Array of Structures, AoS) в об'єктно-орієнтованому підході.
2. **Archetype Chunks** - Дані в ECS організовані в "кусках" (chunks) на основі архетипів, де архетип - це унікальний набір типів компонентів. Всі сутності з однаковим набором компонентів зберігаються поруч у пам'яті.
3. **EntityManager** - Централізований менеджер, що відповідає за створення, модифікацію та знищення сутностей. EntityManager також керує розподілом пам'яті та організацією даних.
4. **Компоненти як дані** - В ECS компоненти є лише структурами даних, без будь-якої логіки. Це дозволяє ефективно упаковувати дані в пам'яті та мінімізує накладні витрати.
5. **Системи як логіка** - Вся логіка гри знаходиться в системах, які обробляють сутності з певними комбінаціями компонентів. Системи можуть бути виконані паралельно, якщо вони не мають конфліктів по даним.

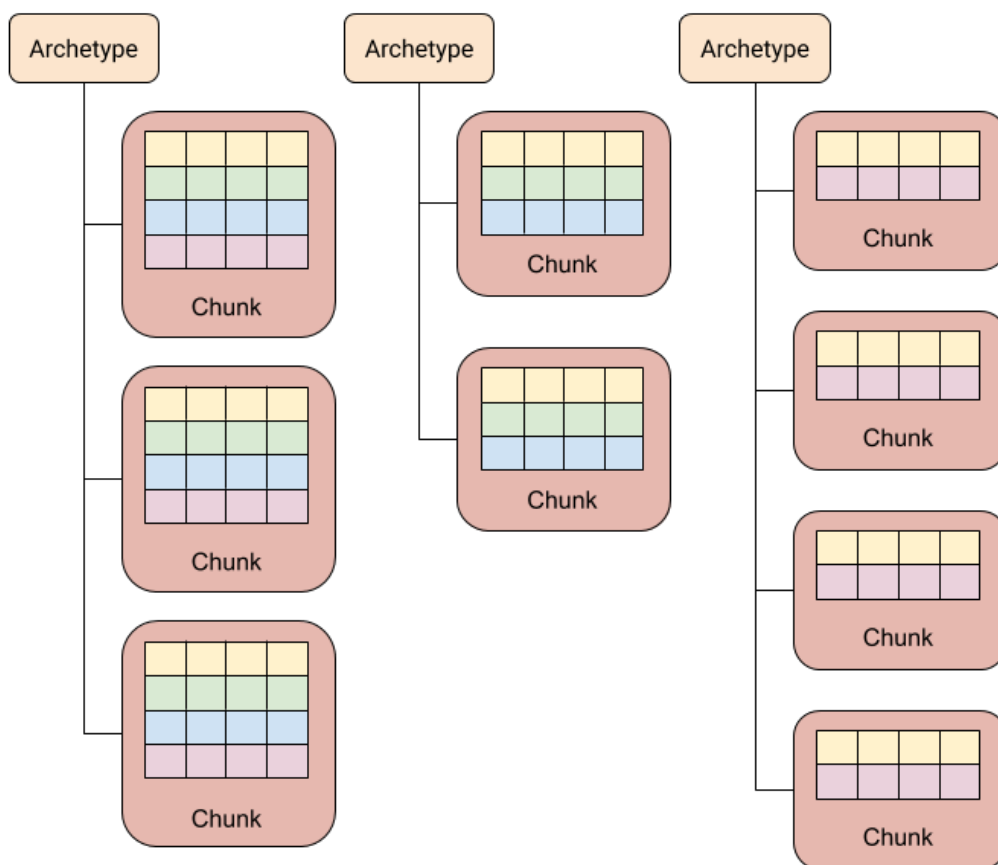


Рис. 2.1. «Візуалізація чанків в Entity-Components-System»

2.2.2. Job System

Job System - це підсистема для організації паралельних обчислень, що дозволяє ефективно розподіляти роботу між доступними ядрами процесора:

1. **Багатопотоковість без блокувань** - Job System використовує архітектуру, що мінімізує блокування та дозволяє максимально використовувати доступні ядра процесора.
2. **Система залежностей** - Автоматичне визначення та керування залежностями між завданнями дозволяє уникнути конфліктів доступу до даних.
3. **Безпека потоків** - Job System забезпечує суворий контроль над доступом до даних, запобігаючи race conditions та інші проблеми багатопотоковості.

4. **Ефективне планування** - Планувальник завдань оптимізує розподіл роботи між потоками для максимальної продуктивності.
5. **Інтеграція з ECS** - Job System тісно інтегрований з ECS через API `.ForEach` та `.Schedule`, що дозволяє легко паралелізувати обробку сутностей.

Job Types:

`IJob` - Одиначне завдання

`IJobParallelFor` - Завдання, що може бути розпаралелене по масиву даних

`IJobForEach` - Спеціалізоване завдання для обробки сутностей в ECS

Scheduling:

`Schedule()` - Планування завдання для асинхронного виконання

`ScheduleParallel()` - Планування завдання для паралельного виконання

`Run()` - Синхронне виконання завдання

Dependency Management:

`JobHandle` - Токен для відстеження залежностей між завданнями

`Complete()` - Очікування завершення завдання

`CombineDependencies()` - Об'єднання залежностей

Рис. 2.2. «Структура Job System»

2.2.3. Burst Compiler

Burst Compiler - це технологія LLVM-компіляції, що перетворює статично типізований код .NET IL у високооптимізований нативний машинний код:

1. **Статичний аналіз** - Burst аналізує код під час компіляції для визначення оптимальних оптимізацій.
2. **SIMD-векторизація** - Автоматичне використання SIMD-інструкцій (Single Instruction, Multiple Data) для паралельної обробки даних.
3. **Розгортання функцій** - Агресивне інлайнінг функцій для зменшення накладних витрат на виклики.
4. **Усунення перевірок безпеки** - Видалення непотрібних перевірок для підвищення продуктивності.

					ІАЛЦ.467200.003 ПЗ	Арк.
						33
Змн.	Арк.	№ докум.	Підпис	Дата		

5. Платформоспецифічні оптимізації - Генерація коду, оптимізованого для конкретної цільової платформи.

Burst працює найефективніше з кодом, що відповідає певним обмеженням:

- Використання структур замість класів
- Відсутність скринькування (boxing) та відскриньковування (unboxing)
- Статичні методи та функції
- Детерміністичні математичні операції
- Обмежене використання рефлексії та віртуальних викликів

2.2.4. Взаємодія компонентів DOTS

Три компоненти DOTS працюють разом, створюючи синергетичний ефект:

1. **ECS організує дані** для ефективного доступу, групує подібні компоненти разом у пам'яті.
2. **Job System розподіляє роботу** між доступними ядрами процесора, забезпечуючи безпечний паралельний доступ до даних.
3. **Burst Compiler оптимізує код** для максимальної продуктивності на цільовій платформі.

Це дозволяє досягти значного підвищення продуктивності порівняно з традиційним підходом, особливо в сценаріях з великою кількістю об'єктів або складними обчисленнями.

2.3. Аналіз продуктивності MonoBehaviour у контексті багатоядерних систем

Традиційний підхід MonoBehaviour в Unity був розроблений в епоху, коли однопотоковість була нормою, а багатоядерні процесори тільки починали з'являтися. З розвитком апаратного забезпечення та збільшенням кількості ядер у процесорах, обмеження цього підходу стали все більш очевидними.

2.3.1. Основні обмеження MonoBehaviour для багатоядерних систем

1. **Монолітний головний потік** - Переважна більшість логіки гри в MonoBehaviour виконується в головному потоці Unity, який відповідає

					ІАЛЦ.467200.003 ПЗ	Арк.
						34
Змн.	Арк.	№ докум.	Підпис	Дата		

за обробку подій життєвого циклу, логіку гри, рендеринг та інші операції. Це створює "вузьке місце" у процесі виконання.

2. **Обмежений доступ до багатопотоковості** - Хоча Unity дозволяє використовувати System.Threading та інші засоби для створення паралельних обчислень, це вимагає ручного управління потоками, синхронізації, та вирішення проблем доступу до даних.
3. **Проблеми з синхронізацією даних** - MonoBehaviour не надає інструментів для безпечного паралельного доступу до даних. Розробники повинні самотійно реалізовувати механізми синхронізації, що призводить до складності та потенційних помилок.
4. **Блокуючі виклики** - Багато операцій в MonoBehaviour є блокуючими, що означає, що вони зупиняють виконання потоку до свого завершення. Це призводить до неефективного використання процесорного часу.

2.3.2. Технічні проблеми для паралельних обчислень

Технічні проблеми паралельних обчислень у MonoBehaviour пов'язані з кількома ключовими обмеженнями. Архітектура Unity з монолітним головним потоком ускладнює розпаралелювання операцій. Доступ до спільних ресурсів з різних потоків потребує додаткових механізмів синхронізації, які створюють накладні витрати, часто нівелюючи переваги паралелізації. Суттєвою проблемою є також недетерміністична поведінка, що виникає при паралельних обчисленнях, ускладнюючи налагодження та тестування.

2.3.3. Проблеми з кешем процесора

Неефективне використання кешу процесора є одним з основних обмежень MonoBehaviour. Розрізнене розміщення об'єктів у пам'яті призводить до частих промахів кешу. Перемішування кодів і даних у єдиних структурах спричиняє конфлікти в кеші. Неоптимальні патерни доступу до пам'яті знижують ефективність передвибірки даних. Додатково, MonoBehaviour компоненти

					ІАЛЦ.467200.003 ПЗ	Арк.
						35
Змн.	Арк.	№ докум.	Підпис	Дата		

часто містять надлишкові дані, що завантажуються в кеш, але не використовуються для конкретних операцій.

2.3.4. Підходи до оптимізації в рамках MonoBehaviour

Розробники використовують різні стратегії для подолання обмежень MonoBehaviour. Пул об'єктів зменшує фрагментацію пам'яті та навантаження на збирач сміття. Coroutines дозволяють розподілити обчислення на кілька кадрів. Ручне розпаралелювання через System.Threading або async/await допомагає перенести обчислення в інші потоки. Оптимізація структур даних із застосуванням масивів та структур замість списків та класів покращує локальність даних. Використання нативного коду або технологій типу IL2CPP підвищує продуктивність критичних ділянок. Проте ці стратегії не вирішують фундаментальних обмежень архітектури.

2.4. Аналіз оптимізацій DOTS для сучасних CPU архітектур

DOTS був розроблений з урахуванням особливостей сучасних процесорних архітектур, з фокусом на максимальне використання можливостей багатоядерних систем та оптимізацію для ефективного використання кешу процесора.

2.4.1. Локальність даних та кеш-оптимізації

Одна з ключових оптимізацій DOTS - це організація даних для максимальної локальності:

1. **Структура масивів (SoA)** - ECS організовує компоненти одного типу послідовно в пам'яті, що дозволяє ефективно завантажувати їх у кеш процесора.
2. **Архетипи та "куски" (chunks)** - Сутності з однаковим набором компонентів (архетипом) зберігаються поруч у пам'яті, в контингентних "кусках" (chunks).

					ІАЛЦ.467200.003 ПЗ	Арк.
						36
Змн.	Арк.	№ докум.	Підпис	Дата		

3. **Компактні структури даних** - Компоненти в ECS зазвичай є невеликими структурами, що містять лише необхідні дані для конкретного аспекту сутності.
4. **Передбачуваний доступ до пам'яті** - Системи в ECS обробляють компоненти послідовно, що дозволяє процесору ефективно передбачати та завантажувати дані заздалегідь.

```
public class MovementSystem : SystemBase
{
    protected override void OnUpdate()
    {
        // Обробка позицій та швидкостей послідовно
        Entities
            .ForEach((ref Translation position, in Velocity velocity) =>
            {
                position.Value += velocity.Value * Time.DeltaTime;
            })
            .Schedule();
    }
}
```

Рис. 2.3. «Приклад системи, що оптимізована для кешу»

2.4.2. Векторизація та SIMD-оптимізації

DOTS активно використовує SIMD-інструкції (Single Instruction, Multiple Data) для паралельної обробки даних на рівні процесорних регістрів:

1. **Автоматична векторизація** - Burst Compiler автоматично векторизує відповідні операції, використовуючи SIMD-інструкції.
2. **Оптимізовані математичні бібліотеки** - Unity.Mathematics надає оптимізовані типи даних (float4, float3x4 тощо) для ефективного використання SIMD.
3. **Пакетна обробка** - ECS дозволяє обробляти кілька сутностей однією SIMD-інструкцією, значно підвищуючи продуктивність для однотипних операцій.

					ІАЛЦ.467200.003 ПЗ	Арк.
						37
Змн.	Арк.	№ докум.	Підпис	Дата		

4. **Специфічні для платформи оптимізації** - Burst Compiler генерує різний код для різних платформ, використовуючи наявні SIMD-розширення (SSE, AVX, NEON тощо).

```
[BurstCompile]
private struct SimulationJob : IJobParallelFor
{
    public void Execute(int index)
    {
        float4 positions = /* ... */;
        float4 velocities = /* ... */;

        // Одна SIMD-інструкція обробляє 4 значення одночасно
        positions += velocities * deltaTime;
    }
}
```

Рис. 2.4. «Приклад використання SIMD-оптимізованої математики»

2.4.3. Механізми паралелізації

DOTS надає потужні інструменти для ефективного розпаралелювання обробки даних:

1. **Job System** - Дозволяє розбивати роботу на невеликі незалежні завдання, які можуть виконуватися паралельно.
2. **Автоматичне управління залежностями** - Job System автоматично відстежує залежності між завданнями та планує їх виконання відповідним чином.
3. **Безпечний доступ до даних** - Система перевіряє конфлікти доступу до даних на етапі компіляції, запобігаючи race conditions.
4. **Різні стратегії планування** - Підтримка різних стратегій планування завдань (Schedule, ScheduleParallel, Run) для різних сценаріїв.

5. Масштабованість - Job System автоматично адаптується до кількості доступних ядер процесора.

```
public class PhysicsSystem : SystemBase
{
    protected override void OnUpdate()
    {
        // Паралельна обробка фізики для кожної сутності
        Entities
            .ForEach((ref PhysicsVelocity velocity, in PhysicsMass mass, in
PhysicsForce force) =>
            {
                velocity.Linear += force.Value / mass.Value * Time.DeltaTime;
            })
            .ScheduleParallel();
    }
}
```

Рис. 2.5. «Приклад паралельного виконання через Job System»

2.4.4. Неблокуючі операції та асинхронність

DOTS сприяє використанню неблокуючих операцій для максимальної утилізації процесора. Завдання плануються для асинхронного виконання за допомогою методів `Schedule` та `ScheduleParallel`. DOTS використовує граф залежностей для планування систем, мінімізуючи блокування. Для ресурсоємних операцій застосовуються спеціалізовані API, що не блокують головний потік під час завантаження ресурсів, фізичних обчислень та інших важких операцій. Архітектура дозволяє створювати складні пайплайни паралельних обчислень, де результати одного завдання передаються наступному без блокування всього процесу.

2.4.5. Оптимізації для різних апаратних платформ

DOTS адаптується до різних апаратних платформ через `Burst Compiler`. Цей компілятор генерує оптимізований код для різних архітектур процесорів, автоматично використовуючи доступні розширення (SSE, AVX, NEON тощо)

					ІАЛЦ.467200.003 ПЗ	Арк.
						39
Змн.	Арк.	№ докум.	Підпис	Дата		

без необхідності писати спеціальний код. Система забезпечує автоматичне балансування між паралелізмом на рівні ядер та SIMD-інструкцій, а також підтримує гетерогенні обчислення з можливістю використання спеціалізованих пристроїв, таких як GPU, для певних завдань. Ці оптимізації дозволяють DOTS ефективно використовувати можливості сучасних процесорів, значно підвищуючи продуктивність порівняно з традиційним підходом MonoBehaviour, особливо в сценаріях з великою кількістю об'єктів або складними обчисленнями.

2.5. Методологія порівняння продуктивності для алгоритму

A*

Для об'єктивного порівняння продуктивності двох підходів до реалізації алгоритму A* (MonoBehaviour та DOTS) необхідно розробити чітку методологію тестування, що забезпечить справедливе порівняння та достовірні результати.

2.5.1. Принципи тестування

Методологія тестування базується на наступних принципах:

- Ідентичність алгоритму** - В обох реалізаціях використовується один і той же базовий алгоритм A*, з однаковими евристиками та функціями оцінки.
- Ідентичність вхідних даних** - Обидві реалізації тестуються на однакових навігаційних сітках та з однаковими початковими та цільовими точками.
- Ізоляція від інших систем** - Тести проводяться в ізольованому середовищі, де відсутній вплив інших систем (рендеринг, фізика тощо).
- Статистична значущість** - Для кожного сценарію тестування проводиться достатня кількість вимірювань для забезпечення статистичної значущості результатів.

					ІАЛЦ.467200.003 ПЗ	Арк.
						40
Змн.	Арк.	№ докум.	Підпис	Дата		

5. **Об'єктивність метрик** - Використовуються чіткі, об'єктивні метрики продуктивності, що вимірюються стандартними інструментами профілювання.

2.6.2. Сценарії тестування

Для комплексної оцінки продуктивності використовуються різні сценарії тестування:

1. **Одиночний пошук шляху** - Вимірювання часу, необхідного для знаходження одного шляху.
2. **Множинний одночасний пошук** - Вимірювання часу, необхідного для знаходження кількох шляхів одночасно (10, 100, 1000 агентів).
3. **Динамічні перешкоди** - Оцінка продуктивності при зміні навігаційної сітки під час виконання (додавання/видалення перешкод).
4. **Складність навігаційної сітки** - Тестування на сітках різної складності (різної розмірності та щільності).
5. **Стрес-тестування** - Оцінка продуктивності в екстремальних умовах (дуже велика кількість агентів, складна навігаційна сітка).

2.6.3. Метрики продуктивності

Для порівняння продуктивності використовуються наступні метрики:

1. **Час виконання** - Середній час, необхідний для завершення алгоритму (мілісекунди).
2. **Використання пам'яті** - Обсяг пам'яті, необхідний для роботи алгоритму (мегабайти).
3. **Масштабованість** - Залежність часу виконання від кількості агентів.
4. **Стабільність продуктивності** - Стандартне відхилення часу виконання, що відображає стабільність продуктивності.
5. **Вплив на частоту кадрів** - Вплив виконання алгоритму на загальну частоту кадрів (FPS).

					ІАЛЦ.467200.003 ПЗ	Арк.
						41
Змн.	Арк.	№ докум.	Підпис	Дата		

6. **Використання процесора** - Завантаження процесора під час виконання алгоритму.

2.6.4. Інструменти профілювання

Для збору метрик використовуються наступні інструменти:

1. **Unity Profiler** - Для загального профілювання продуктивності.
2. **Custom Profiling Markers** - Для детального профілювання окремих частин алгоритму.
3. **Memory Profiler** - Для аналізу використання пам'яті.
4. **Burst Inspector** - Для аналізу коду, згенерованого Burst Compiler.
5. **Системні лічильники продуктивності** - Для моніторингу загального стану системи.

2.6.5. Методика проведення тестів

Кожен тест проводиться за наступною методикою:

1. **Підготовка середовища** - Створення тестової сцени з необхідними компонентами.
2. **Калібрування** - Проведення кількох "розігрівальних" запусків для стабілізації стану системи.
3. **Збір даних** - Проведення серії тестів з фіксацією всіх метрик.
4. **Статистична обробка** - Обчислення середніх значень, стандартних відхилень та інших статистичних показників.
5. **Візуалізація результатів** - Представлення результатів у вигляді графіків та таблиць.
6. **Аналіз та інтерпретація** - Формулювання висновків на основі отриманих даних.

					ІАЛЦ.467200.003 ПЗ	Арк.
						42
Змн.	Арк.	№ докум.	Підпис	Дата		

2.6.6. Контроль чинників, що впливають на результати

Для забезпечення об'єктивності результатів необхідно контролювати наступні чинники:

1. **Апаратне забезпечення** - Всі тести проводяться на одному і тому ж комп'ютері.
2. **Навантаження системи** - Під час тестів відсутні фонові процеси, що могли б вплинути на результати.
3. **Версія Unity** - Використовується одна і та ж версія Unity для всіх тестів.
4. **Налаштування проекту** - Ідентичні налаштування компіляції, оптимізації та інших параметрів проекту.
5. **Конфігурація тестів** - Ідентичні параметри алгоритму для обох реалізацій.

Ця методологія дозволить отримати об'єктивні дані про продуктивність двох підходів до реалізації алгоритму A* та зробити обґрунтовані висновки щодо їх переваг та недоліків.

2.7. Критерії вибору архітектури для різних типів проектів

Вибір між архітектурою MonoBehaviour та DOTween є важливим архітектурним рішенням, яке має значний вплив на розробку та продуктивність гри. Для прийняття обґрунтованого рішення необхідно враховувати ряд критеріїв, що відповідають специфіці конкретного проекту.

					ІАЛЦ.467200.003 ПЗ	Арк.
						43
Змн.	Арк.	№ докум.	Підпис	Дата		

2.7.1. Технічні критерії

Таблиця 2.1. Технічні Критерії для Mono та DOTS

Критерій	MonoBehaviour	DOTS
Масштаб та складність проекту	Підходить для невеликих проектів з обмеженою кількістю об'єктів	Рекомендується для великих проектів з тисячами об'єктів або складними симуляціями
Вимоги до продуктивності	Підходить для проектів, де продуктивність не є критичною	Рекомендується для проектів з високими вимогами до частоти кадрів (60+ FPS)
Цільові платформи	Достатній для високопродуктивних платформ (ПК, консолі)	Необхідний для мобільних платформ та пристроїв з обмеженими ресурсами
Багатоядерність цільових пристроїв	Підходить для пристроїв з одним або двома ядрами	Ефективно використовує всі доступні ресурси на пристроях з багатьма ядрами
Тип гри та ігрова механіка	Підходить для ігор з меншою кількістю об'єктів, але складною логікою	Рекомендується для ігор з великою кількістю об'єктів (стратегії, симулятори, шутери)

2.7.2. Проектні критерії

Таблиця 2.2. Проектні критерії для Mono та DOTS

Критерій	MonoBehaviour	DOTS
Стадія розробки	Підходить для існуючих проектів, де перехід на DOTS може бути складним і ризикованим	Стратегічно виправданий вибір для нових проектів
Розмір та досвід команди	Краще для менших команд з меншим досвідом через простоту розробки	Ефективніше використовується більшими командами з досвідченими розробниками
Часові обмеження	Забезпечує швидший старт при жорстких термінах розробки	Інвестиції окупаються в довгостроковій перспективі при довших термінах розробки
Бюджет проекту	Підходить для менших проектів з обмеженим бюджетом	Виправданий для більших проектів з більшим бюджетом для досягнення кращої продуктивності

2.7.3. Технологічні критерії

Таблиця 2.3. Технологічні критерії для Mono та DOTS

Критерій	MonoBehaviour	DOTS
Інтеграція з іншими системами	Краще при використанні багатьох сторонніх плагінів, які не підтримують DOTS	Простіша інтеграція, якщо проект використовує власні системи
Мережеві функції	Достатній для одиночних ігор	Надає переваги в детермінізмі та продуктивності для мережевих ігор
Вимоги до фізики та пошуку шляху	Достатній, якщо ці функції не є центральними для гри	Надає значні переваги для ігор, що вимагають складної фізики або пошуку шляху для багатьох об'єктів

2.7.4. Матриця рішень для різних типів проектів

Для спрощення процесу прийняття рішень можна використовувати наступну матрицю, яка рекомендує підхід на основі ключових характеристик проекту:

Таблиця 2.4. Матриця рішень для різних типів проектів Mono та DOTS

Тип проекту	Рекомендований підхід	Обґрунтування
Мобільна казуальна гра	MonoBehaviour	Простота розробки, відносно невелика кількість об'єктів
Мобільна стратегія з тисячами юнітів	DOTS	Висока продуктивність при великій кількості об'єктів
ПК/консольний шутер	Гібридний	DOTS для критичних систем, MonoBehaviour для іншого
ПК/консольна RPG	MonoBehaviour	Складна логіка, але відносно невелика кількість одночасних об'єктів
VR/AR додаток	DOTS	Високі вимоги до продуктивності для досягнення комфортного FPS
Симулятор з реалістичною фізикою	DOTS	Ефективна обробка складних фізичних взаємодій
Багатокористувацька онлайн-гра	DOTS	Детермінізм та ефективна синхронізація стану
Освітній додаток з простою графікою	MonoBehaviour	Простота розробки, низькі вимоги до продуктивності

2.7.5. Підхід до прийняття рішення

Для обґрунтованого вибору архітектури необхідно дотримуватись поетапного підходу. Спочатку проводиться аналіз вимог проєкту щодо продуктивності, масштабу та складності. Далі оцінюються доступні ресурси: час, бюджет та досвід команди. Важливим етапом є прототипування, яке дозволяє порівняти підходи на практиці. Після цього аналізуються потенційні ризики для кожного рішення. На основі отриманих даних приймається фінальне рішення, що найкраще відповідає потребам проєкту.

Важливо пам'ятати, що вибір не обов'язково має бути бінарним. Гібридний підхід, де DOTS використовується для критичних для продуктивності систем, а MonoBehaviour - для іншої функціональності, часто є оптимальним рішенням для багатьох проєктів.

					ІАЛЦ.467200.003 ПЗ	Арк.
						48
Змн.	Арк.	№ докум.	Підпис	Дата		

ВИСНОВОК ДО РОЗДІЛУ 2

У цьому розділі було проведено детальний аналіз архітектури MonoBehaviour та DOTS, їх особливостей, переваг та обмежень. Було розглянуто технічні аспекти обох підходів, їх продуктивність у контексті багатоядерних систем та сучасних CPU архітектур.

Основні висновки:

1. **Архітектурні відмінності** - MonoBehaviour базується на об'єктно-орієнтованій парадигмі з інкапсуляцією даних та поведінки, тоді як DOTS використовує дата-орієнтований підхід з чітким розділенням даних та логіки.
2. **Продуктивність** - DOTS демонструє значно кращу продуктивність у сценаріях з великою кількістю об'єктів або складними обчисленнями завдяки ефективному використанню кешу процесора, паралельним обчисленням та оптимізаціям на рівні машинного коду.
3. **Використання багатоядерних систем** - MonoBehaviour має обмежені можливості для паралелізації, тоді як DOTS ефективно розподіля

					ІАЛЦ.467200.003 ПЗ	Арк.
						49
Змн.	Арк.	№ докум.	Підпис	Дата		

РОЗДІЛ 3: РОЗРОБКА СИСТЕМИ ПОРІВНЯЛЬНОГО ТЕСТУВАННЯ

3.1. Архітектура системи тестування

3.1.1. Загальна структура проєкту та принципи організації

Система порівняльного тестування була розроблена як єдиний Unity проєкт, що містить дві повністю ізольовані реалізації алгоритму A*.

Архітектурне рішення базувалося на принципі максимального розділення компонентів при збереженні спільної основи для забезпечення справедливого порівняння.

Структура проєкту організована навколо двох основних гілок функціональності. Перша гілка містить всі компоненти, пов'язані з DOTS реалізацією, включаючи ECS компоненти, системи та job-структури. Друга гілка включає традиційні MonoBehaviour скрипти з відповідною ієрархією класів та менеджерів. Така організація дозволяє розробникам легко орієнтуватися в кодовій базі та забезпечує відсутність випадкових взаємодій між реалізаціями.

Кожна реалізація має власну сцену з попередньо налаштованими об'єктами та компонентами. Сцена DOTSPathfinding містить Entity Manager, системи пошуку шляху та відповідний користувацький інтерфейс. Сцена MonoPathfinding включає традиційні GameObject з MonoBehaviour компонентами та аналогічний UI. Перемикання між сценами відбувається через кнопки інтерфейсу, що дозволяє швидко порівнювати поведінку обох підходів в ідентичних умовах.

					ІАЛЦ.467200.003 ПЗ	Арк.
						50
Змн.	Арк.	№ докум.	Підпис	Дата		

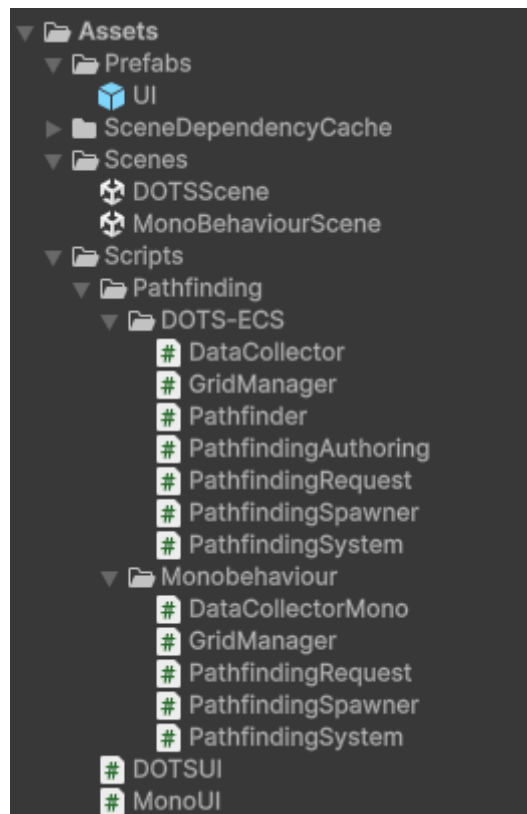


Рис. 3.1. «Структура папок проекту з розділенням DOTS та MonoBehaviour компонентів»

3.1.2. Спільні компоненти

Кілька ключових компонентів використовуються обома реалізаціями для забезпечення ідентичності умов тестування. GridManager відповідає за створення та управління навігаційною сіткою, забезпечуючи однакові умови для обох підходів.

```
private int gridVersion = 0;
public int GridVersion => gridVersion;

public void RegenerateGrid()
{
    InitializeGrid();
    gridVersion++; // Критично для синхронізації
}
```

Рис. 3.2. «Система версіонування сітки»

Коли користувач змінює розмір сітки або відсоток перешкод, GridManager регенерує сітку та збільшує номер версії. Обидві реалізації перевіряють цю версію через кешовану змінну та оновлюють свої внутрішні представлення при необхідності.

Форматування експорту DataCollector:

- **Minimal** - базові метрики (успіх/неуспіх, час виконання)
- **WithCoordinates** - включає координати знайденого шляху
- **WithMaps** - ASCII візуалізація з позначенням шляху

```
public enum ExportFormat
{
    Minimal,           // Тільки success/fail, timing
    WithCoordinates,  // Включає координати як string
    WithMaps           // Включає ASCII карти
}
```

Рис. 3.3. «Формати експорту»

Система автоматично адаптує структуру JSON експорту залежно від обраного формату, оптимізуючи розмір файлів для специфічних потреб аналізу.

3.1.3. Ізоляція реалізацій для справедливого порівняння

Забезпечення справедливого порівняння вимагало ретельної ізоляції між DOTS та MonoBehaviour реалізаціями. Кожна реалізація працює в окремому контексті виконання без взаємного впливу на продуктивність або поведінку іншої. Це досягається через використання окремих сцен, де кожна сцена містить лише компоненти, необхідні для відповідного підходу.

Синхронізація даних між реалізаціями здійснюється через спільні компоненти, які забезпечують ідентичність початкових умов. GridManager гарантує, що обидва підходи використовують абсолютно однакову навігаційну сітку з ідентичним розміщенням перешкод. Система також забезпечує синхронізацію початкових та цільових позицій для пошуку шляху, що критично важливо для об'єктивного порівняння результатів.

Механізм перемикання між реалізаціями реалізований через SceneManager Unity, що дозволяє повністю вивантажувати одну реалізацію

					ІАЛЦ.467200.003 ПЗ	Арк.
						52
Змн.	Арк.	№ докум.	Підпис	Дата		

перед завантаженням іншої. Це запобігає будь-якому залишковому впливу попередньої реалізації на результати тестування та забезпечує чистоту експерименту.

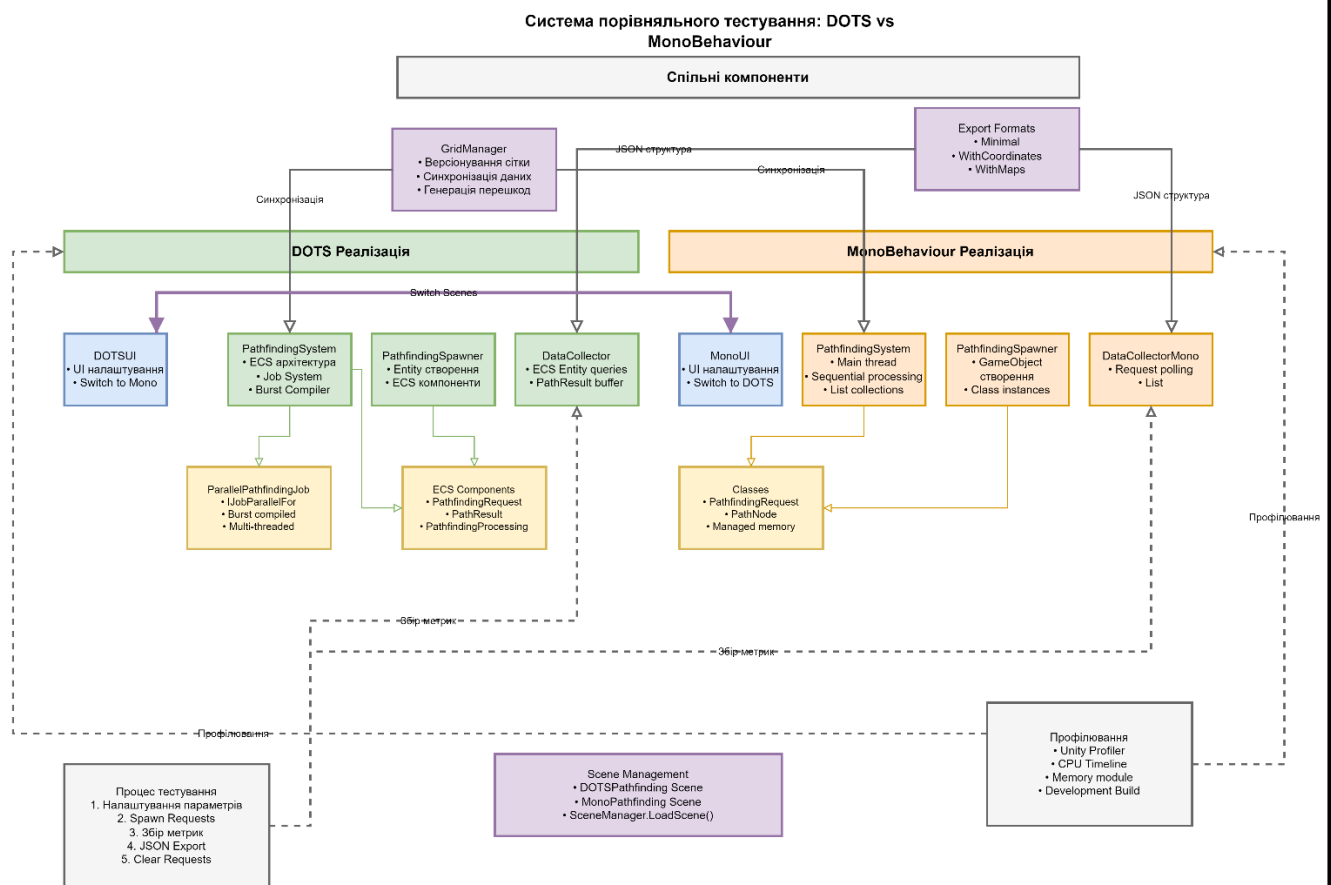


Рис. 3.4. «Діаграма взаємодії компонентів між DOTS та MonoBehaviour реалізаціями»

3.2. Реалізація A* алгоритму в обох підходах

3.2.1. Специфіка DOTS імплементації

DOTS реалізація алгоритму A* побудована навколо Entity Component System архітектури з використанням Job System для паралелізації обчислень.

Основні компоненти ECS:

- **PathfindingRequest** - параметри запиту (start, target, стан обробки)
- **PathResult** - буферний компонент для результатів шляху
- **PathfindingProcessing** - тег-компонент для контролю стану


```
[BurstCompile]
public struct ParallelPathfindingJob : IJobParallelFor
{
    [ReadOnly] public NativeArray<bool> walkableGrid;
    [ReadOnly] public int2 gridSize;
    [ReadOnly] public NativeArray<PathfindingJobData> requests;
    [WriteOnly] public NativeArray<PathfindingJobResult> results;

    public void Execute(int index)
    {
        var request = requests[index];
        if (!request.shouldProcess) return;

        var path = FindPath(request.startPosition, request.targetPosition);
        results[index] = path;
    }
}
```

Рис. 3.5. «ParallelPathfindingJob в DOTS імплементації»

Технічні обмеження:

- **FixedList4096Bytes** - максимум 512 координат на шлях
- **Чанки по 128 сутностей** - мінімум для ефективної паралелізації
- **Обрізання довгих шляхів** - автоматичне позначення через wasTruncated

```
public struct PathfindingJobResult
{
    public int pathLength;
    public bool wasTruncated; // Відстеження обрізання
    public FixedList4096Bytes<int2> pathPositions; // 512 max координат
}
```

Рис. 3.6. «PathfindingJobResult в DOTS імплементації»

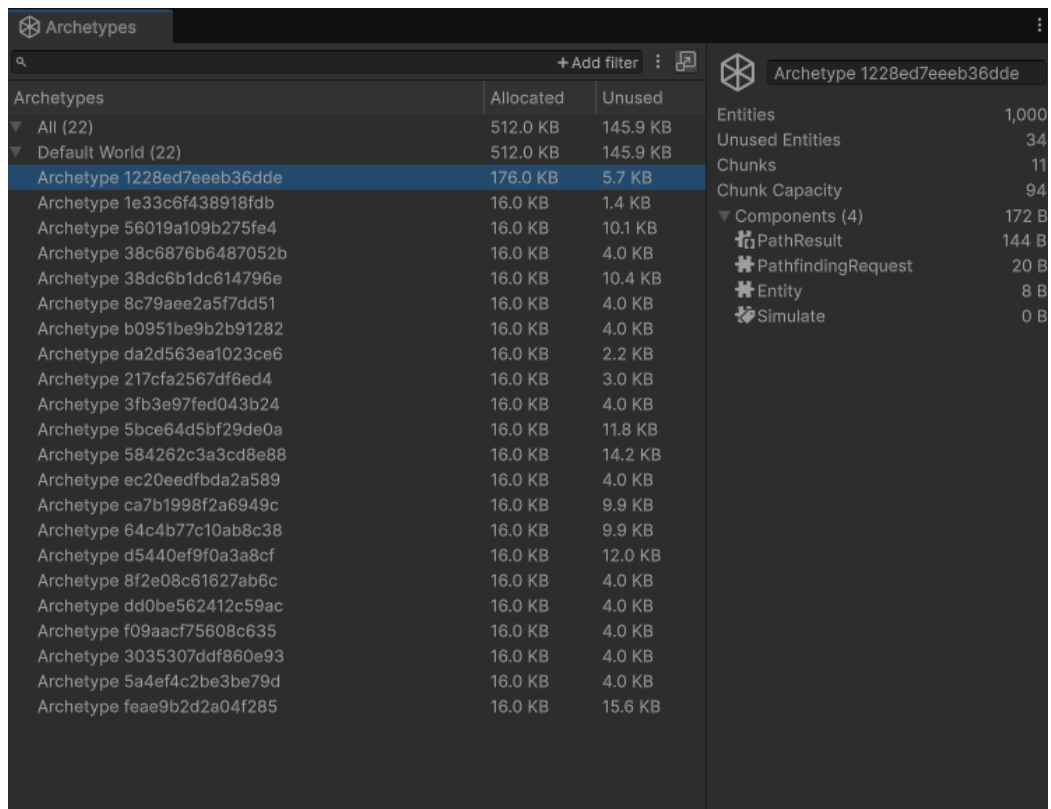


Рис. 3.7. «Entity Debugger з відображенням архетипів та чанків»

3.2.2. Специфіка MonoBehaviour імплементації

MonoBehaviour реалізація використовує традиційний об'єктно-орієнтований підхід з централізованим PathfindingSystem.

Архітектура класів:

- **PathfindingSystem** - центральний менеджер обробки запитів
- **PathfindingRequest** - клас з інкапсуляцією даних та результатів
- **GridManager** - управління навігаційною сіткою

```
[Serializable]
public class PathfindingRequest
{
    public int id;
    public int2 startPosition;
    public int2 targetPosition;
    public bool isProcessing;
    public bool hasResult;
    public List<int2> pathPositions = new List<int2>();
    public bool success;
    public float processingTime;
    public bool wasTruncated;
}
```

Рис. 3.8. «PathfindingRequest в Mono імплементації»

Оптимізації для зменшення GC тиску:

- **Пулінг об'єктів** - повторне використання List<PathNode> структур
- **Кешування результатів** - уникнення повторних обчислень
- **Обмеження ітерацій** - захист від нескінченних циклів

```
private List<int2> FindPath(int2 start, int2 target)
{
    var openSet = new List<PathNode>();
    var closedSet = new HashSet<int2>();

    int iterations = 0;
    int maxIterations = gridSize.x * gridSize.y; // Захист від зависання

    while (openSet.Count > 0 && iterations < maxIterations)
    {
        iterations++;
        // Основна логіка A*
    }

    return result;
}
```

Рис. 3.9. «FindPath в Mono імплементації»

3.2.3. Забезпечення ідентичності алгоритму

Критично важливим аспектом справедливого порівняння є забезпечення ідентичності базового алгоритму A* в обох реалізаціях.

Спільні параметри алгоритму:

- **Евристична функція** - Manhattan distance: $|x_1 - x_2| + |y_1 - y_2|$
- **Вартість переміщення** - 10 для прямих, 14 для діагональних
- **Логіка вибору вузла** - мінімальна f-вартість, потім h-вартість
- **Умови завершення** - досягнення цілі або вичерпання варіантів

					ІАЛЦ.467200.003 ПЗ	Арк.
						56
Змн.	Арк.	№ докум.	Підпис	Дата		

```
// Ідентична евристика в обох реалізаціях
private int CalculateDistance(int2 a, int2 b)
{
    int dx = math.abs(a.x - b.x);
    int dy = math.abs(a.y - b.y);
    return dx > dy ? 14 * dy + 10 * (dx - dy) : 14 * dx + 10 * (dy - dx);
}
```

Рис. 3.10. «Розрахунок евристичної функції»

Відмінності в реалізації:

- **DOTS** - NativeArray, FixedList, unmanaged memory
- **MonoBehaviour** - List<T>, HashSet<T>, managed memory
- **Логіка алгоритму** - абсолютно ідентична

```
// Спільна логіка A* (псевдокод)
while (openSet.HasElements && iterations < maxIterations)
{
    currentNode = GetLowestFCostNode(openSet);
    if (currentNode.position.Equals(target)) break;

    foreach (neighbor in GetNeighbors(currentNode))
    {
        if (!IsWalkable(neighbor) || InClosedSet(neighbor)) continue;

        int tentativeGCost = currentNode.gCost + GetMoveCost(currentNode,
neighbor);
        // Стандартна логіка A*
    }
}
```

Рис. 3.11. «Спільна логіка A* (псевдокод)»

3.3. Система збору метрик та мануального тестування

3.3.1. Unity Profiler налаштування для build версії

Точне вимірювання продуктивності вимагає проведення тестів на зібраній версії застосунку, а не в Unity Editor. Редактор створює значний накладний вплив через системи відлагодження, інспектори та інші допоміжні компоненти, що може спотворювати реальну картину продуктивності обох підходів.

Конфігурація збірки включає активацію Development Build режиму з увімкненою опцією Autoconnect Profiler. Це дозволяє Unity Profiler

					ІАЛЦ.467200.003 ПЗ	Арк.
						57
Змн.	Арк.	№ докум.	Підпис	Дата		

автоматично підключатися до запущеного застосунку без необхідності ручного налаштування з'єднання. Додатково активується Script Debugging для можливості відлагодження зібраної версії.

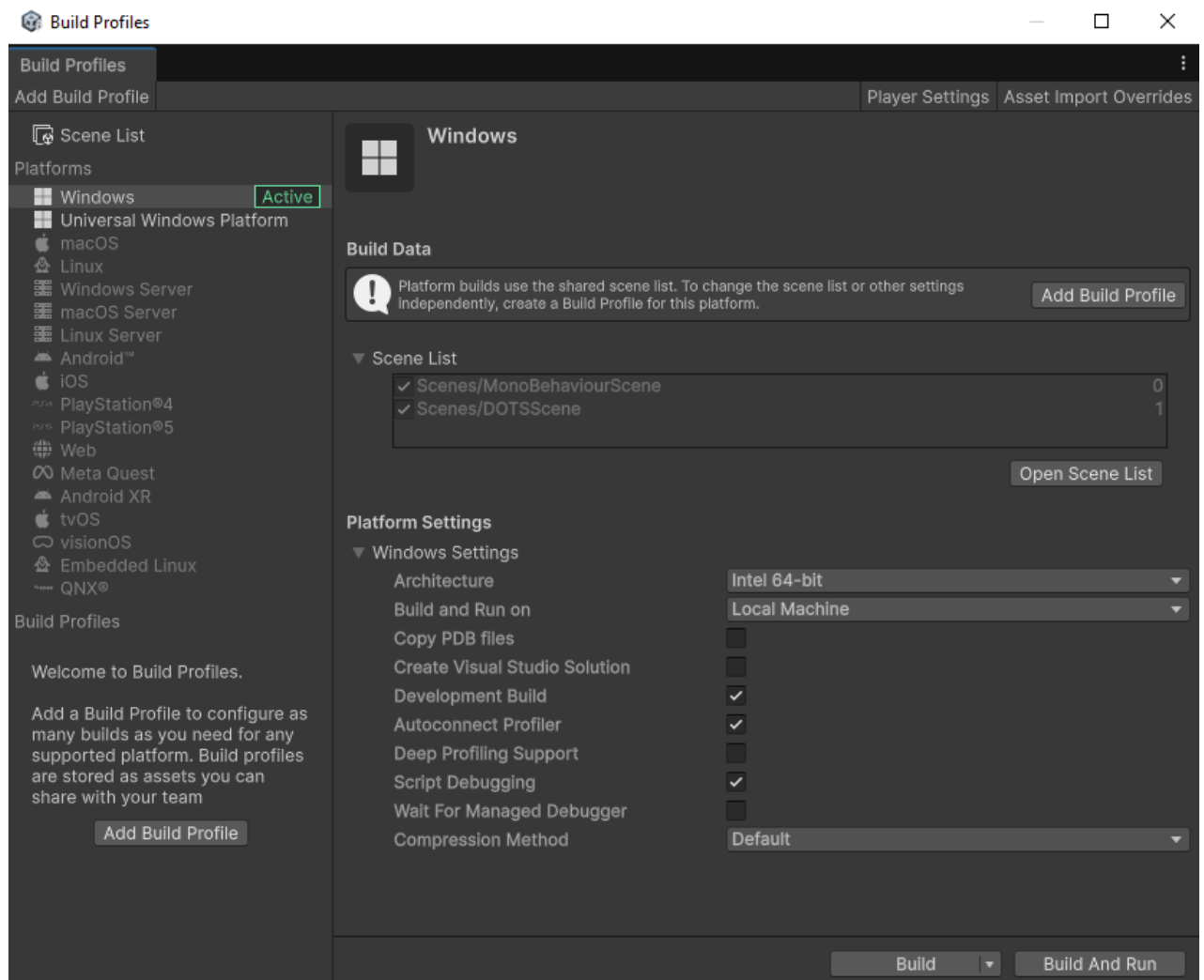


Рис. 3.12. «Підключення профайлеру до зібраної версії»

Основні модулі Profiler для збору метрик:

- **CPU Usage** - детальна інформація про час виконання методів та систем
- **Memory** - використання heap memory та частота garbage collection
- **Rendering** - вплив на частоту кадрів та GPU навантаження

Profiler налаштовується з активними модулями CPU Usage та Memory для збору ключових метрик продуктивності. CPU Timeline надає детальну інформацію про час виконання різних методів та систем, що дозволяє точно виміряти час роботи алгоритму A* в обох реалізаціях. Memory module

					ІАЛЦ.467200.003 ПЗ	Арк.
						58
Змн.	Арк.	№ докум.	Підпис	Дата		

відстежує використання пам'яті та частоту garbage collection, що особливо важливо для порівняння MonoBehaviour підходу з DOTS.

3.3.2. Збір метрик через DataCollector

DataCollector система автоматично збирає метрики, які можуть бути отримані програмно без залежності від Unity Profiler.

Структура зібраних метрик:

- **Часові метрики** - час виконання алгоритму через Stopwatch (мілісекунди)
- **Метрики якості** - відсоток успішних пошуків, довжина шляхів
- **Метрики успішності** - кількість truncated paths, завершених пошуків
- **Системні метрики** - FPS через Time.unscaledDeltaTime

```
[System.Serializable]
public class PathRecord
{
    public int id;
    public int2 start;
    public int2 end;
    public bool success;
    public int length;
    public float timeMs; // Виміряно через System.Diagnostics.Stopwatch
    public string coordinates; // Опціонально для WithCoordinates
    public List<string> map; // Опціонально для WithMaps
}
```

Рис. 3.13. «Запис шляху в JSON формат»

```
private void ProcessPathfindingRequestImmediate(PathfindingRequest request)
{
    var startTime = System.Diagnostics.Stopwatch.StartNew();

    request.isProcessing = true;
    var path = FindPath(request.startPosition, request.targetPosition);

    request.processingTime = (float)startTime.Elapsed.TotalMilliseconds;
    request.hasResult = true;
    request.isProcessing = false;
}
```

Рис. 3.14. «Вимірювання часу виконання»

					ІАЛЦ.467200.003 ПЗ	Арк.
						59
Змн.	Арк.	№ докум.	Підпис	Дата		

```
// Критична перевірка для DOTS реалізації
bool pathComplete = false;
bool pathTruncated = false;

if (pathBuffer.Length > 0)
{
    var lastPosition = pathBuffer[pathBuffer.Length - 1].position;
    pathComplete = lastPosition.Equals(request.targetPosition);

    // Перевірка обрізання через ліміт буфера
    if (!pathComplete && pathBuffer.Length >= 63)
    {
        pathTruncated = true;
        truncatedPaths++;
    }
}

record.success = pathComplete && !pathTruncated; // Тільки повні шляхи
```

Рис. 3.15. «Відстеження обрізаних шляхів в DOTS»

Автоматичний експорт при досягненні ліміту:

- **Максимум шляхів** - користувацька настройка (за замовчуванням 100)
- **Автоекспорт** - кожні 30 секунд або при заповненні буфера
- **JSON структура** - timestamp, gridSize, paths[], stats{}

Важливо зазначити, що детальні метрики пам'яті (heap usage, GC.Alloc) можуть бути отримані лише через Unity Profiler під час мануального аналізу.

3.3.3. Мануальна процедура тестування

Мануальне тестування проводиться за стандартизованою процедурою для забезпечення консистентності результатів.

Послідовність дій для кожного тесту:

1. **Підготовка** - запуск Development Build з Autoconnect Profiler
2. **Налаштування** - встановлення параметрів через UI (сітка, агенти, перешкоди)
3. **Виконання** - запуск генерації запитів кнопкою "Spawn Requests"
4. **Моніторинг** - відстеження прогресу через Performance та Statistics UI
5. **Збір даних** - автоматичний або ручний експорт JSON результатів
6. **Очищення** - кнопка "Clear Requests" для видалення невиконаних запитів

					ІАЛЦ.467200.003 ПЗ	Арк.
						60
Змн.	Арк.	№ докум.	Підпис	Дата		

```
// Автоматичне збереження при досягненні ліміту
if (currentData.paths.Count >= maxPaths)
{
    if (autoExport)
    {
        ExportData();
        InitializeData(); // Скидання для продовження
    }
    return;
}
```

Рис. 3.16. «Автоматичне збереження при досягненні ліміту»

```
public void ClearAllRequests()
{
    // Очищення лише невиконаних запитів, не завершених результатів
    var query = entityManager.CreateEntityQuery(typeof(PathfindingRequest));
    var entities = query.ToEntityArray(Unity.Collections.Allocator.Temp);

    Debug.Log($"Clearing {entities.Length} pending pathfinding requests...");
    entityManager.DestroyEntity(entities);
    entities.Dispose();
}
```

Рис. 3.17. «Функція Clear Requests»

Стандартизовані тест-кейси:

Сценарій 1: Базове тестування

- Розмір сітки: 100×100
- Кількість агентів: 1, 10, 100, 500, 1000
- Відсоток перешкод: 20%

Сценарій 2: Масштабування сітки

- Розміри сітки: 100×100, 250×250, 500×500
- Кількість агентів: 1000
- Відсоток перешкод: 25%

Сценарій 3: Вплив перешкод

- Розмір сітки: 100×100
- Кількість агентів: 1000
- Відсоток перешкод: 20%, 30%, 50%, 70%

					ІАЛЦ.467200.003 ПЗ	Арк.
						61
Змн.	Арк.	№ докум.	Підпис	Дата		

Сценарій 4: Стрес-тестування

- Розмір сітки: 500×500
- Кількість агентів: 2000, 3000, 5000
- Відсоток перешкод: 75%

Важливо зазначити, що кнопка "Clear Requests" очищає тільки активні невиконані запити (pending requests), а не завершені результати пошуку. Завершені шляхи залишаються в DataCollector до експорту або досягнення максимального ліміту.

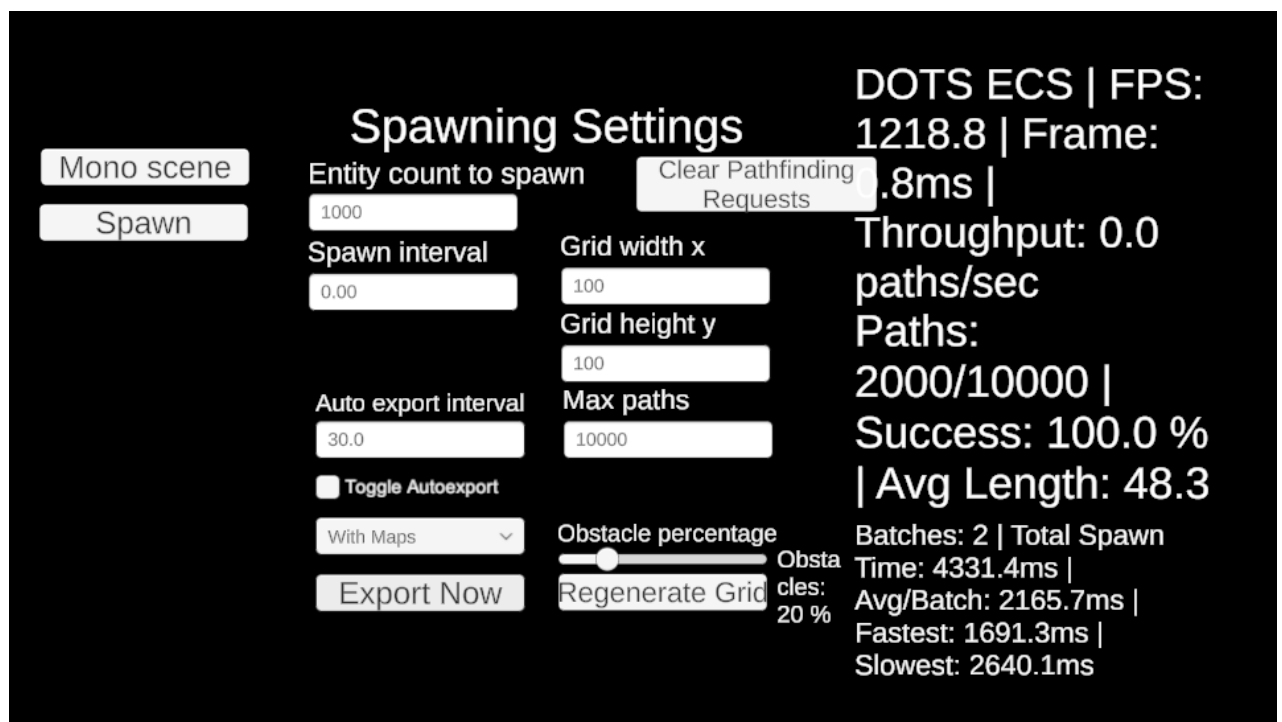


Рис. 3.18. «Скріншот процедури мануального тестування з активним UI та Profiler (1)»

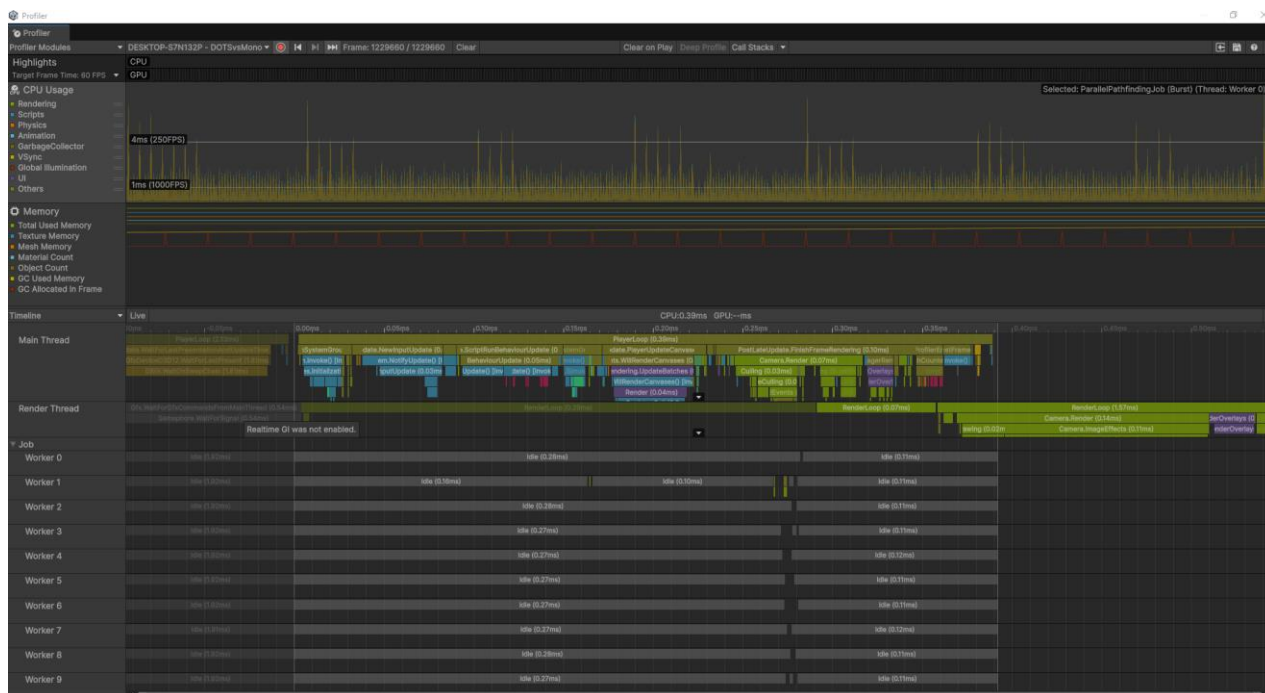


Рис. 3.19. «Скріншот процедури мануального тестування з активним UI та Profiler (2)»

3.4. Обробка та візуалізація даних тестування

3.4.1. Конвертація JSON в CSV формат

Експортовані з Unity дані у JSON форматі потребують конвертації в табличний формат для аналізу та створення візуалізацій. Структура JSON файлів включає вкладені об'єкти з інформацією про окремі пошуки шляхів, загальну статистику та метадані про умови тестування.

Змн.	Арк.	№ докум.	Підпис	Дата

ІАЛЦ.467200.003 ПЗ

Арк.

63

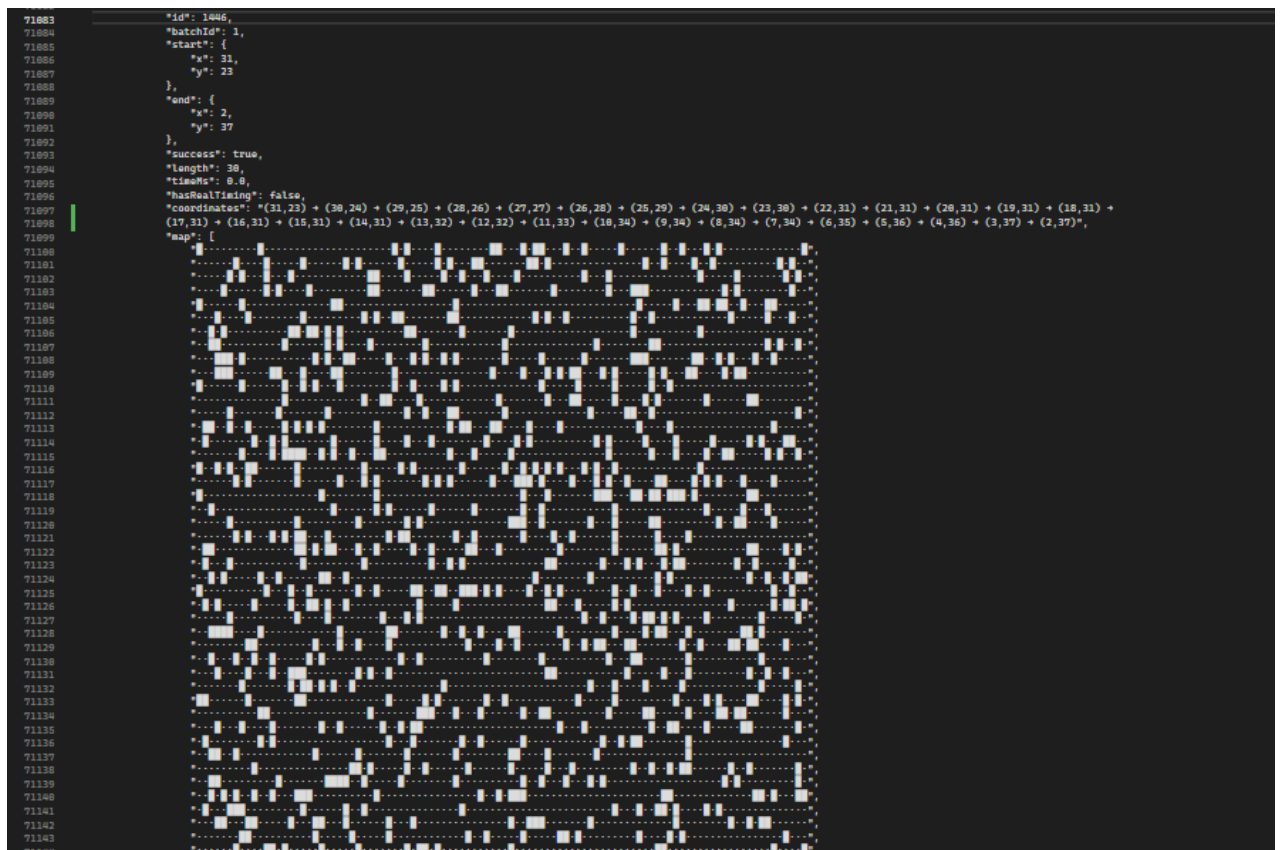


Рис. 3.20. «Скріншот прикладу структури json файлу з мапою»

3.4.3. Аналіз результатів через візуалізацію

Комплексний аналіз результатів базується на порівнянні декількох ключових аспектів продуктивності через спеціалізовані візуалізації. Dashboard підхід дозволяє одночасно відображати різні метрики та швидко ідентифікувати тренди та аномалії в даних.

Аналіз ефективності використання пам'яті проводиться через порівняння кількості garbage collection викликів та піків використання heap memory. DOTS реалізація зазвичай демонструє значно менше GC активності через використання unmanaged memory, що відображається в стабільніших графіках споживання пам'яті.

Оцінка якості алгоритму включає аналіз відсотка успішних пошуків та середньої довжини знайдених шляхів.

3.5. Користувацький інтерфейс системи

3.5.1. UI компоненти для налаштування

Користувацький інтерфейс системи тестування розроблений як єдиний набір компонентів, що забезпечують повний контроль над параметрами тестування та моніторингом результатів. DOTSUI та MonoUI компоненти надають ідентичний функціонал для забезпечення справедливого порівняння можливостей налаштування обох підходів.

Секція налаштувань спавнера дозволяє контролювати процес генерації запитів на пошук шляху. Користувач може встановити кількість сутностей для одночасного тестування від одиниць до тисяч агентів, налаштувати інтервал між створенням запитів для контролю навантаження системи та використовувати кнопки для швидкого запуску або очищення всіх активних запитів.

Конфігурація збирача даних включає налаштування максимальної кількості шляхів для збору в одному тестовому циклі, параметри автоматичного експорту з можливістю встановлення інтервалу та вибір формату експорту даних залежно від потреб аналізу. Мінімальний формат зберігає лише базові метрики, розширений включає координати шляхів, а повний додає ASCII візуалізацію карт.

Управління навігаційною сіткою здійснюється через налаштування розмірів сітки, слайдер для контролю відсотка перешкод та кнопку регенерації сітки для створення нових тестових умов. Всі зміни параметрів сітки автоматично синхронізуються між DOTS та MonoBehaviour реалізаціями.

					ІАЛЦ.467200.003 ПЗ	Арк.
						65
Змн.	Арк.	№ докум.	Підпис	Дата		

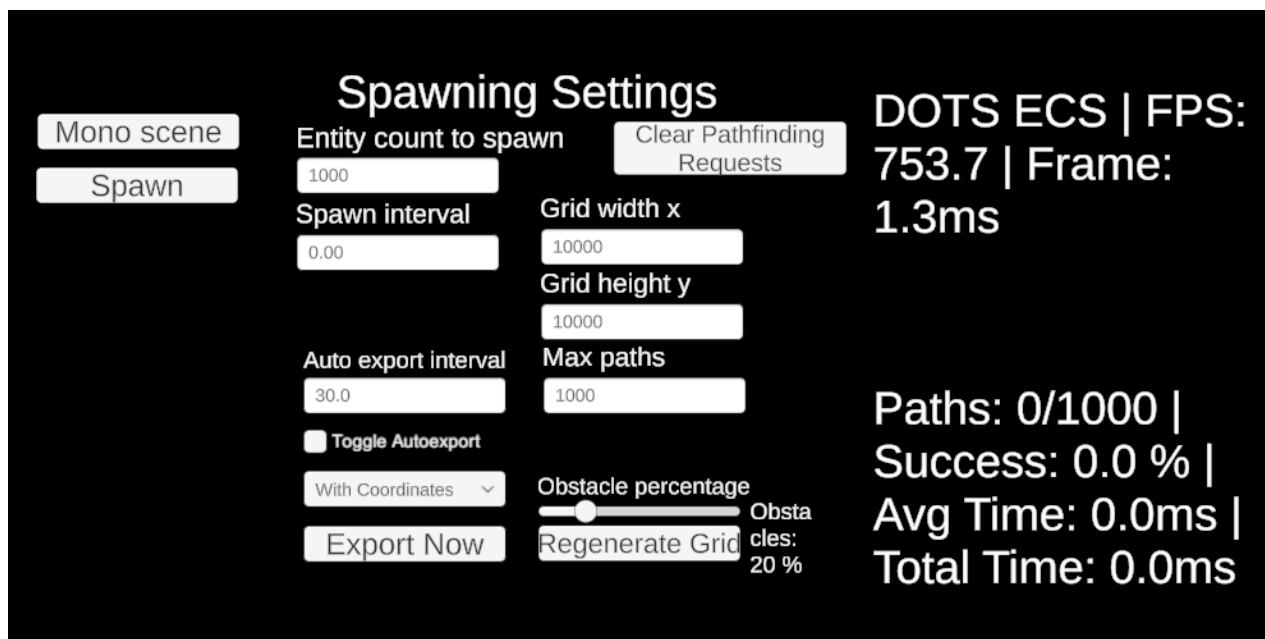


Рис. 3.21. «Скріншот повного користувацького інтерфейсу з усіма секціями налаштувань»

3.5.2. Перемикання між реалізаціями

Система перемикання між DOTS та MonoBehaviour реалізаціями реалізована через SceneManager Unity з кнопками швидкого переходу в інтерфейсі кожної реалізації. Кнопка "Switch to MonoBehaviour" в DOTS сцені та "Switch to DOTS" в MonoBehaviour сцені забезпечують миттєвий перехід без втрати налаштувань.

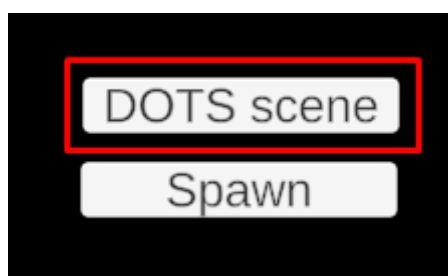


Рис. 3.22. «Кнопки перемикання між реалізаціями в UI»

3.5.3. Відображення поточних результатів

Система реального часу відображення результатів забезпечує миттєвий зворотний зв'язок про стан тестування та поточну продуктивність системи. Performance display показує поточний FPS та час кадру з оновленням кожні 30 кадрів для стабільності відображення. Додатково відображається ідентифікатор поточної реалізації для швидкої орієнтації.

					ІАЛЦ.467200.003 ПЗ	Арк.
						66
Змн.	Арк.	№ докум.	Підпис	Дата		

Statistics display надає інформацію про поточний прогрес збору даних, включаючи кількість зібраних шляхів відносно максимального ліміту, відсоток успішних пошуків та середній час виконання алгоритму. Ця інформація оновлюється в реальному часі при завершенні кожного пошуку шляху.



Рис. 3.23. «Панель відображення результатів в реальному часі»

3.6. Технічні особливості та обмеження

3.6.1. Обмеження DOTS реалізації

DOTS реалізація має кілька технічних обмежень, які впливають на можливості тестування та інтерпретацію результатів. Найсуттєвішим є ліміт FixedList4096Bytes на 512 координат для зберігання результатів пошуку. Це обмеження проявляється при тестуванні на великих сітках або при пошуку довгих шляхів, коли алгоритм знаходить маршрут довший за доступний буфер.

					ІАЛЦ.467200.003 ПЗ	Арк.
						67
Змн.	Арк.	№ докум.	Підпис	Дата		

Система автоматично відстежує випадки обрізання через флаг `wasTruncated` в `PathfindingJobResult` та позначає такі шляхи як неуспішні для коректної статистики. `DataCollector` додає індикатор "[TRUNCATED]" до координат обрізаних шляхів при експорті в розширеному форматі, що дозволяє ідентифікувати проблемні випадки під час аналізу.

Організація пам'яті в ECS створює додаткові особливості для тестування. Unity групує сутності в чанки по 128 елементів з однаковим архетипом компонентів. При кількості сутностей менше 128 Job System може не активувати паралелізацію, виконуючи всі обчислення в одному потоці. Це означає, що переваги DOTS стають помітними лише при тестуванні з достатньою кількістю одночасних запитів.

3.6.2. Результати проведених тестів

Тестування системи проводилося на комп'ютері з AMD Ryzen 7 9700X (8 ядер, 16 потоків) та 64GB RAM. Всі тести виконувалися на зібраній версії застосунку з підключеним Unity Profiler.

Тест-кейс 1: Базове порівняння продуктивності

Перший тест-кейс порівнював час виконання алгоритму A* для різної кількості одночасних агентів на сітці 100×100 з 20% перешкод. Тестувалися конфігурації з 1, 10, 100, 500 та 1000 агентів для виявлення точки.

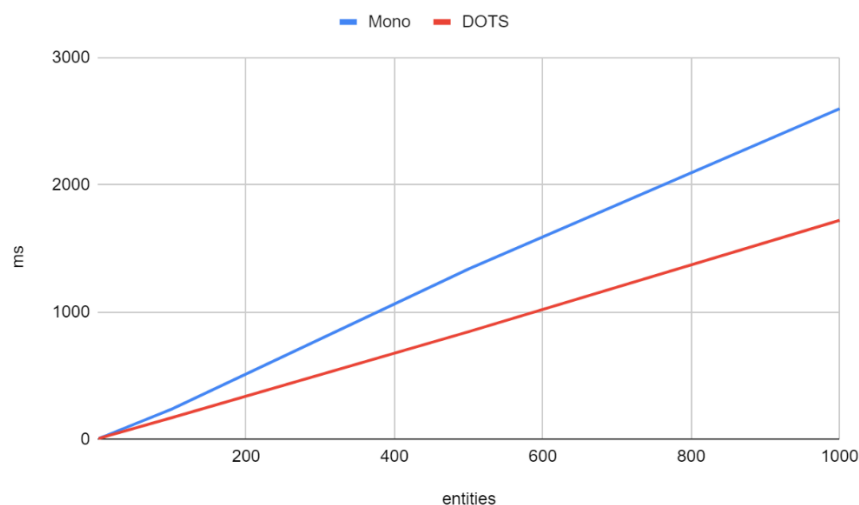


Рис. 3.24. «Діаграма швидкості обчислень відносно кількості сутностей»

Результати показали що продуктивність в нашому випадку росте лінійно з ростом кількості сутностей на сітці 100x100. DOTS має невелику перевагу. Unity Profiler демонструє кардинально різні патерни використання CPU. MonoBehaviour показує послідовні піки навантаження на одному потоці, тоді як DOTS розподіляє навантаження рівномірно між усіма доступними ядрами процесора.

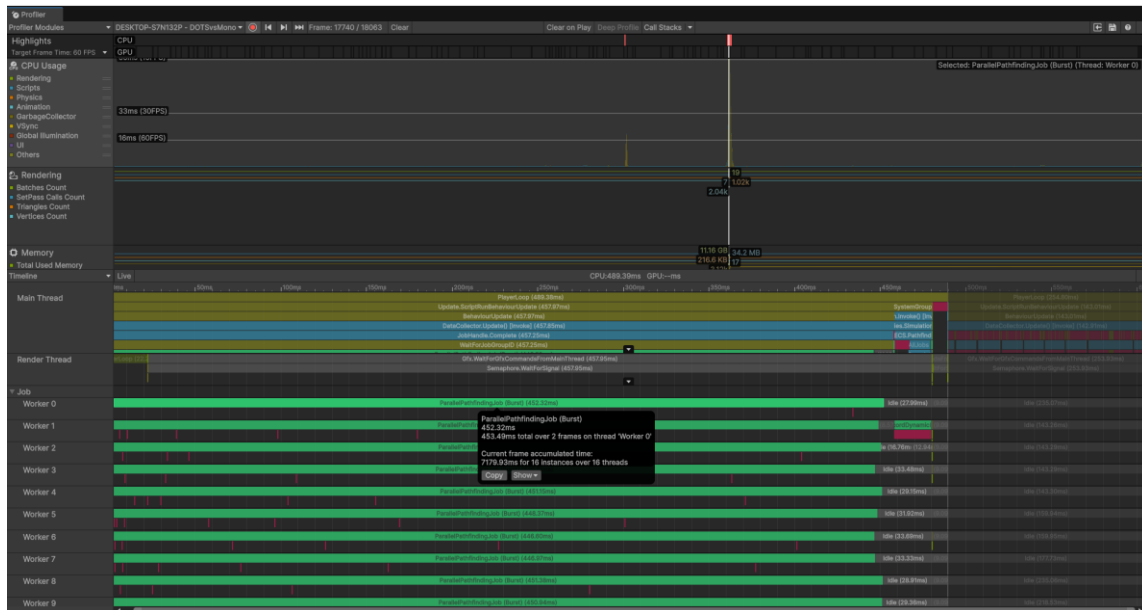


Рис. 3.25. «Unity Profiler CPU Timeline під час виконання 1000 агентів DOTS»

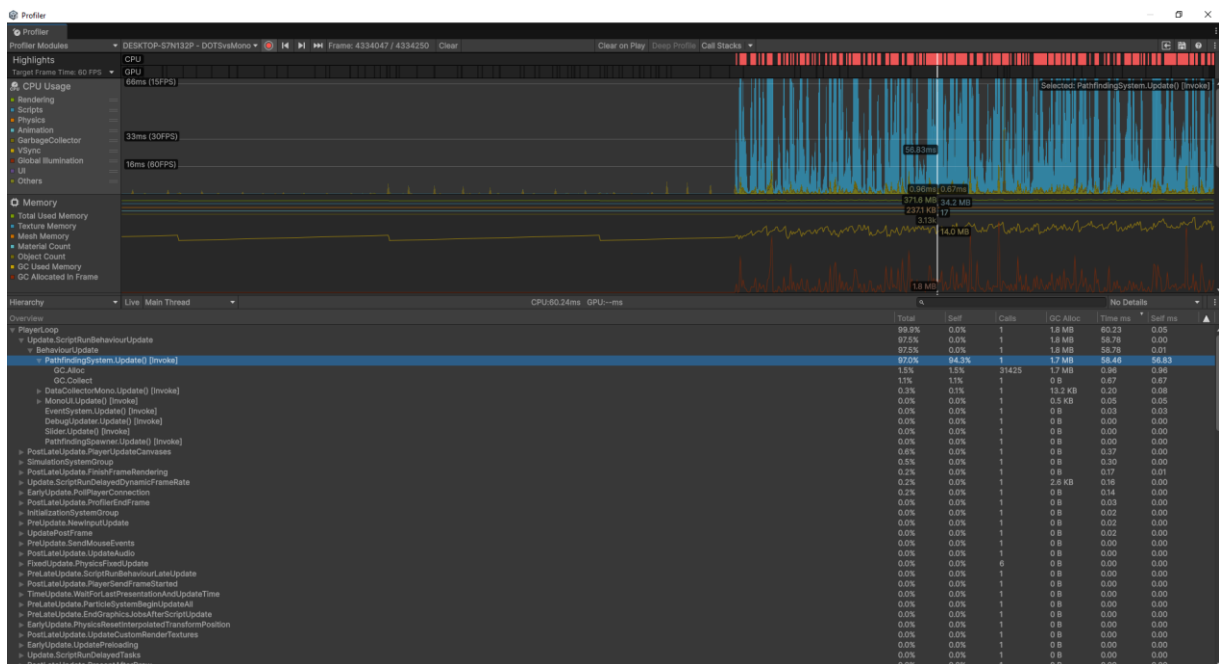


Рис. 3.26. «Unity Profiler CPU Timeline під час виконання 1000 агентів MonoBehaviour»

Тест-кейс 2: Масштабування розміру сітки

Другий тест-кейс досліджував вплив розміру навігаційної сітки на продуктивність обох підходів. Тестування проводилося з фіксованою кількістю 1000 агентів на сітках розміром 100×100 , 250×250 та 500×500 клітинок з постійним 25% перешкод.

Результати виявили нелінійне експоненційне зростання часу виконання для MonoBehaviour. DOTS демонструє більш лінійне масштабування з коефіцієнтом, що вказує на кращу оптимізацію для роботи з великими наборами даних.

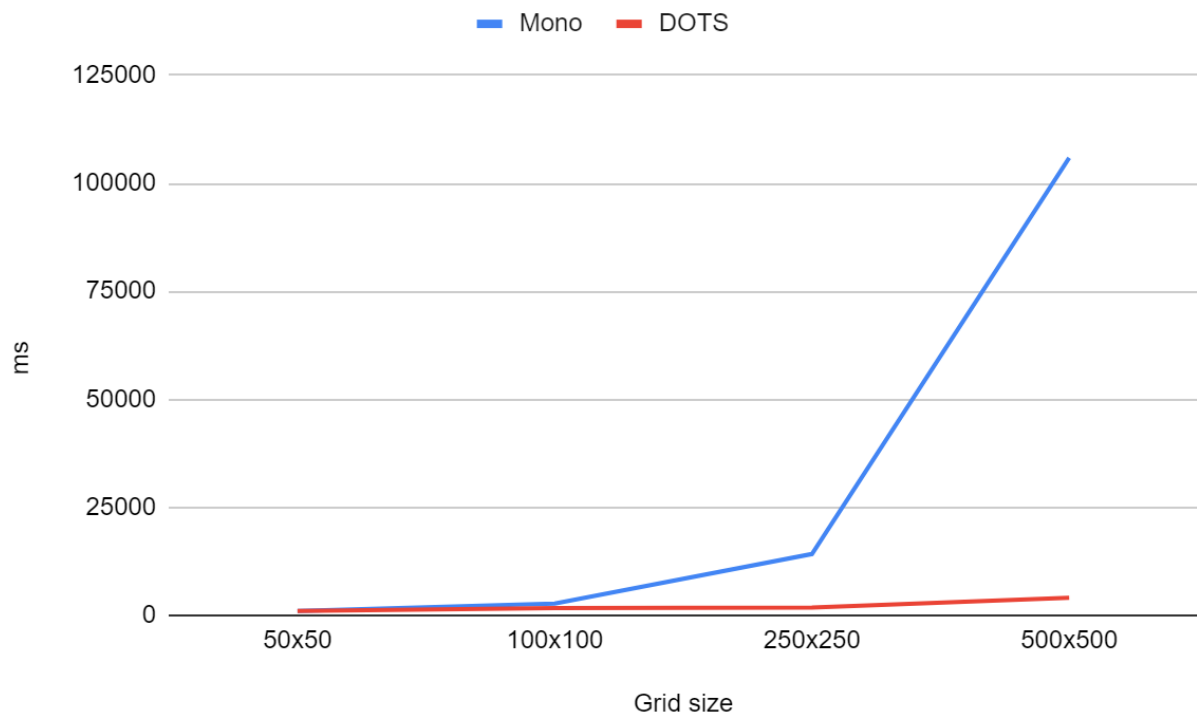


Рис. 3.27. «Лінійний графік масштабування відносно розміру сітки»

Тест-кейс 3: Вплив щільності перешкод

Третій тест-кейс аналізував вплив відсотка перешкод на складність пошуку та продуктивність алгоритму. Тестування проводилося на сітці 100×100 з 1000 агентів при щільності перешкод 20%, 30%, 50% та 70%.

Результати показали очікуване зростання складності пошуку зі збільшенням кількості перешкод. MonoBehaviour демонструє більш різке зростання часу

					ІАЛЦ.467200.003 ПЗ	Арк.
						70
Змн.	Арк.	№ докум.	Підпис	Дата		

виконання при високій щільності перешкод через неефективне використання кешу при обході складних областей. DOTS зберігає більш стабільну продуктивність завдяки оптимізованому доступу до даних.

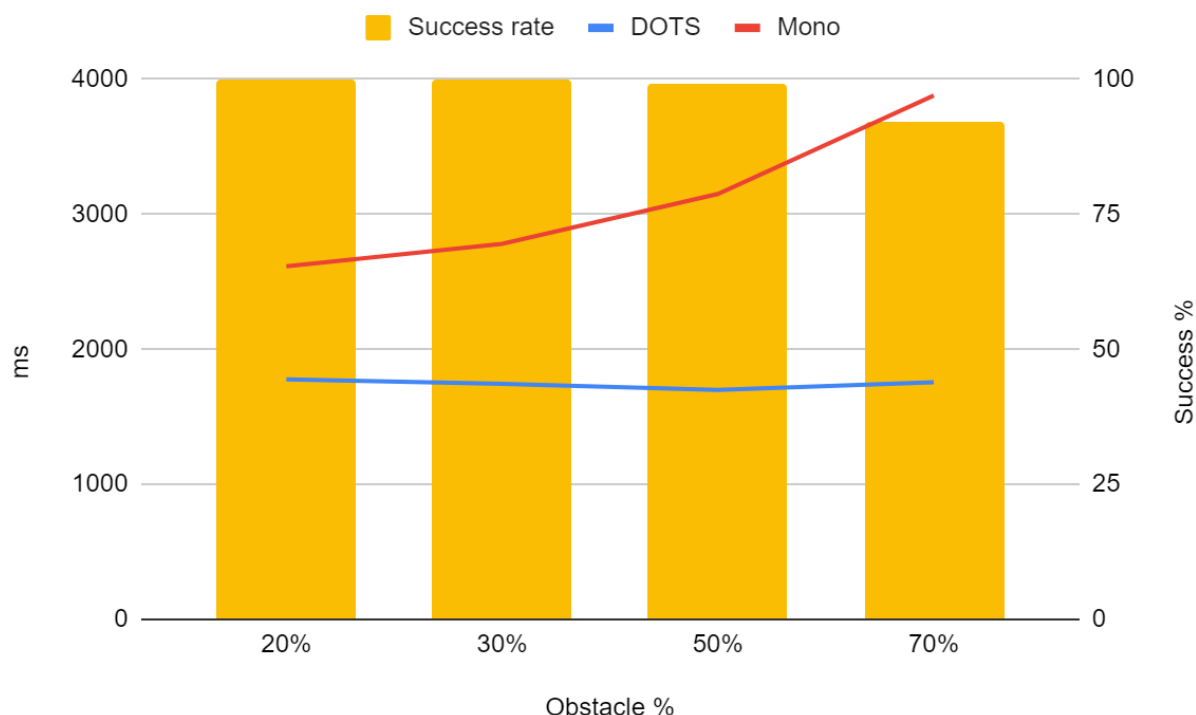


Рис. 3.28. «Комбінований графік часу виконання та відсотка успіху відносно щільності перешкод»

Тест-кейс 4: Аналіз використання пам'яті

Четвертий тест-кейс зосереджувався на порівнянні патернів використання пам'яті між підходами. Тестування проводилося з 1000 агентів на сітці 500×500 з 50% кількості перешкод з моніторингом heap memory.

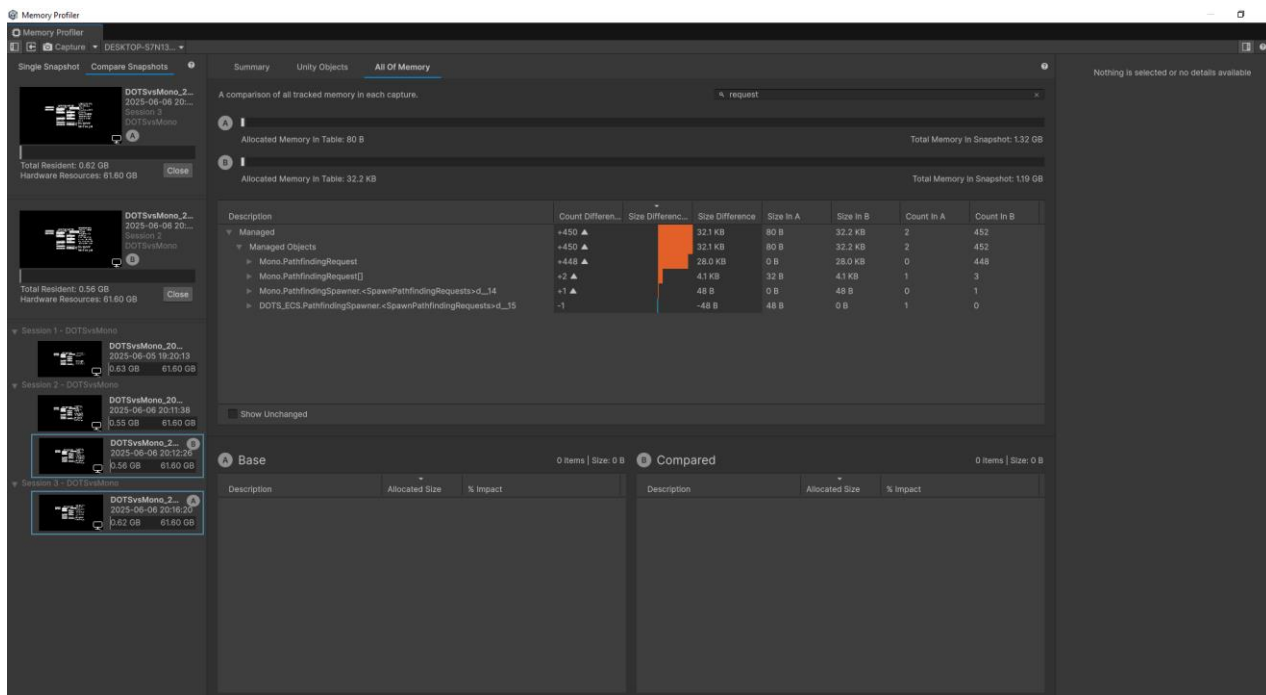


Рис. 3.29. «Unity Profiler Memory usage порівняння DOTS vs MonoBehaviour»

Unmanaged memory використання в DOTS нажаль ми неможемо засікти використовуючи профайлер, але з моїх тестів об'єкти настільки малі що всю різницю в використанні даних можна просто привести як статистичну похибку.

Тест-кейс 5: Стрес-тестування граничних умов

П'ятий тест-кейс перевіряв поведінку обох систем при екстремальних навантаженнях. Тестування включало 2000, 3000 та 5000 одночасних агентів на сітці 500×500 для виявлення точок відмови та деградації продуктивності.

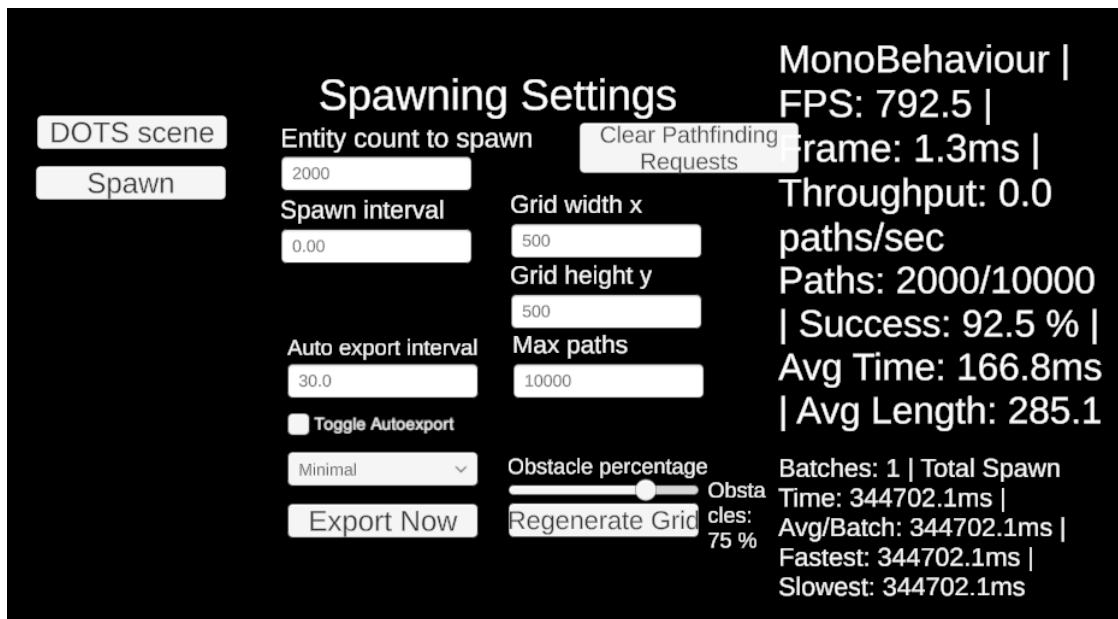


Рис. 3.30. «Стресс-тест MonoBehaviour імплементациі»

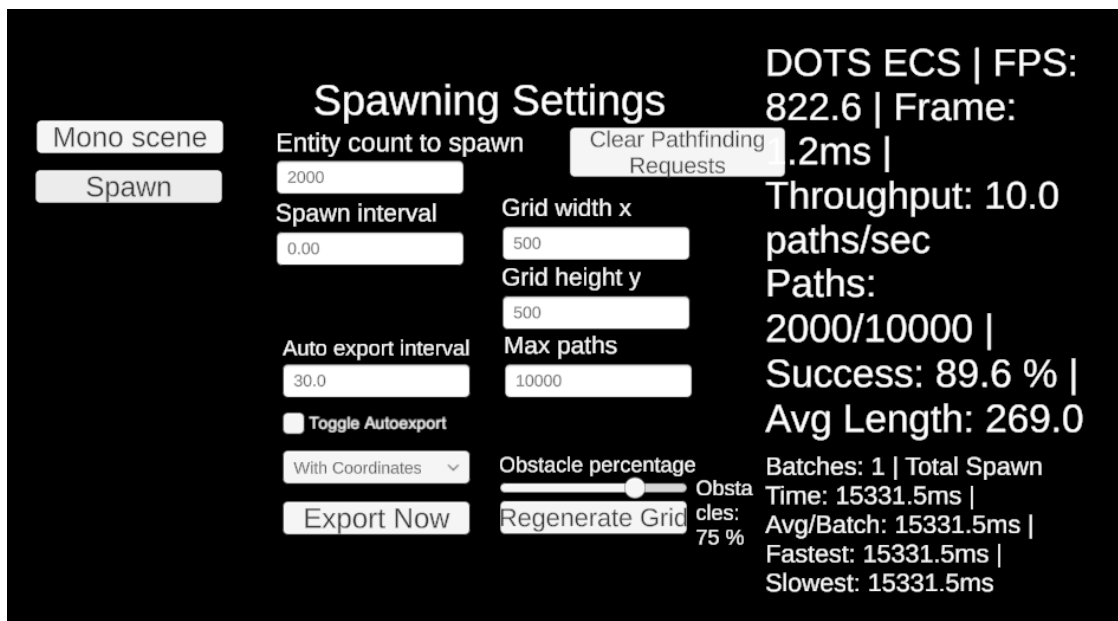


Рис. 3.31. «Стресс-тест DOTS імплементациі»

З цих стресс тестів ніякої нової інформації не було отримано, продуктивність реалізованої імплементациі Monobehaviour починає сильно деградувати від збільшення розміру сітки, а просто збільшення сутностей немає настільки великого впливу.

ВИСНОВОК ДО РОЗДІЛУ 3

У цьому розділі було розроблено та імплементовано комплексну систему порівняльного тестування продуктивності DOTS та MonoBehaviour підходів для алгоритму пошуку шляху A*.

Основні досягнення:

1. **Архітектура системи** - створено єдиний Unity проєкт з двома повністю ізольованими реалізаціями алгоритму A*, що забезпечує справедливе порівняння при збереженні ідентичності базового алгоритму.
2. **Технічна реалізація** - DOTS імплементация використовує ECS архітектуру з Job System для паралелізації, тоді як MonoBehaviour реалізація базується на традиційному об'єктно-орієнтованому підході. Обидві реалізації використовують ідентичну евристичну функцію та логіку A*.
3. **Система збору метрик** - розроблено DataCollector для автоматичного збору часових метрик та статистики якості, з підтримкою експорту в JSON форматі та можливістю аналізу через Unity Profiler.
4. **Результати тестування** - проведено п'ять тест-кейсів, що показали лінійне масштабування DOTS та експоненційне зростання часу виконання MonoBehaviour при збільшенні розміру сітки. DOTS демонструє ефективніше використання багатоядерних процесорів та більш стабільну продуктивність при високій щільності перешкод.
5. **Технічні обмеження** - виявлено ліміт FixedList4096Bytes (512 координат) для DOTS реалізації та необхідність мінімум 128 сутностей для активації паралелізації Job System.

Розроблена система надає об'єктивний інструментарій для порівняння архітектурних підходів Unity та може бути використана для прийняття обґрунтованих рішень щодо вибору технології залежно від специфіки проєкту.

					ІАЛЦ.467200.003 ПЗ	Арк.
						74
Змн.	Арк.	№ докум.	Підпис	Дата		

ВИСНОВКИ

Дане дослідження було присвячено порівняльному аналізу продуктивності двох фундаментально різних підходів до розробки в Unity: традиційного об'єктно-орієнтованого MonoBehaviour та сучасного дата-орієнтованого DOTS (Data-Oriented Technology Stack). Основною метою роботи було дослідження можливостей та переваг DOTS системи через практичну імплементацію алгоритму пошуку шляху A*.

Теоретичні досягнення:

1. **Систематизація знань** - проведено комплексний аналіз еволюції архітектури обчислювальних систем та їх впливу на парадигми програмування, що дозволило виявити передумови створення DOTS як відповіді на обмеження традиційного підходу.
2. **Архітектурний аналіз** - детально розглянуто внутрішню структуру обох підходів, їх переваги та обмеження в контексті сучасних багатоядерних систем, що надає теоретичну основу для прийняття архітектурних рішень.
3. **Методологія порівняння** - розроблено комплексну методологію для об'єктивного порівняння продуктивності різних архітектурних підходів, включаючи критерії вибору для різних типів проєктів.

Практичні результати:

1. **Система тестування** - створено повнофункціональну систему порівняльного тестування з двома ізольованими реалізаціями алгоритму A*, що забезпечує справедливе порівняння при збереженні ідентичності базового алгоритму.
2. **Емпіричні дані** - отримано конкретні метрики продуктивності, що демонструють переваги DOTS у сценаріях масштабування та роботи з великими наборами даних, підтверджуючи теоретичні припущення про ефективність дата-орієнтованого підходу.

					ІАЛЦ.467200.003 ПЗ	Арк.
						75
Змн.	Арк.	№ докум.	Підпис	Дата		

3. **Практичні рекомендації** - сформульовано критерії вибору архітектури для різних типів проєктів, що має безпосередню практичну цінність для розробників Unity.

Обмеження дослідження:

Важливо визнати, що дана робота має певні обмеження, які впливають на інтерпретацію результатів:

1. **Специфічність імплементації** - розроблені реалізації можуть не представляти оптимальних варіантів для кожного з підходів. Існує вірогідність значного покращення продуктивності обох реалізацій через застосування додаткових оптимізацій та кращих практик програмування.
2. **Обмеженість тест-кейсів** - використані сценарії тестування, хоча і покривають основні аспекти продуктивності, можуть недостатньо відображати складність та різноманітність реальних застосунків Unity. Реальні ігрові проєкти включають множину взаємодіючих систем, складні ієрархії об'єктів та різноманітні патерни використання.
3. **Синтетичний характер тестів** - ізольоване тестування алгоритму A* не враховує інтеграцію з іншими системами Unity (рендеринг, фізика, аудіо, UI), що є критично важливим в реальних умовах розробки.

Напрямки подальших досліджень:

1. **Гібридні підходи** - дослідження ефективності комбінування DOTS для критичних систем з MonoBehaviour для менш ресурсоемних компонентів.
2. **Реальні кейси** - тестування на повнофункціональних ігрових проєктах з урахуванням взаємодії всіх підсистем Unity.
3. **Платформоспецифічні оптимізації** - аналіз поведінки обох підходів на різних цільових платформах (мобільні пристрої, консолі, VR/AR).

Загальний висновок:

Дослідження підтвердило, що DOTS представляє значний крок вперед у розвитку архітектури Unity, особливо для проєктів з високими вимогами до

					ІАЛЦ.467200.003 ПЗ	Арк.
						76
Змн.	Арк.	№ докум.	Підпис	Дата		

продуктивності та масштабованості. Однак вибір між DOTS та MonoBehaviour повинен базуватися на комплексному аналізі вимог конкретного проєкту, досвіду команди та довгострокових цілей розробки.

Традиційний підхід MonoBehaviour залишається життєздатним та ефективним для широкого спектра застосунків, особливо тих, де простота розробки та швидкість прототипування є пріоритетними. DOTS, у свою чергу, відкриває нові можливості для створення високопродуктивних застосунків, але вимагає більш глибокого розуміння принципів дата-орієнтованого програмування.

Результати цієї роботи надають розробникам об'єктивну основу для прийняття обґрунтованих архітектурних рішень та сприяють кращому розумінню можливостей обох підходів у контексті сучасної розробки ігор та інтерактивних застосунків.

					ІАЛЦ.467200.003 ПЗ	Арк.
						77
Змн.	Арк.	№ докум.	Підпис	Дата		

СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

1. Stroustrup, B. What is Object-Oriented Programming? [Електронний ресурс] – Режим доступу до ресурсу: <https://dl.acm.org/doi/10.1109/52.2020>.
2. Unity Technologies Unity ECS Documentation [Електронний ресурс] – Режим доступу до ресурсу: <https://docs.unity3d.com/Packages/com.unity.entities@1.2/manual/ecs-packages.html>.
3. Unity Technologies DOTS - Unity's Data-Oriented Technology Stack [Електронний ресурс] – Режим доступу до ресурсу: <https://unity.com/dots>.
4. Unity Technologies C# Job System Overview [Електронний ресурс] – Режим доступу до ресурсу: <https://docs.unity3d.com/6000.0/Documentation/Manual/job-system-overview.html>.
5. Thompson, J. (TurboMakesGames) Unity DOTS Tutorial Series [Електронний ресурс] – Режим доступу до ресурсу: <https://youtube.com/TurboMakesGames>.
6. Lasa, H. (Code Monkey) Unity DOTS Complete Course [Електронний ресурс] – Режим доступу до ресурсу: <https://youtube.com/c/CodeMonkeyUnity>.
7. Unity Forum DOTS is awesome, but what will happen to MonoBehaviours? [Електронний ресурс] – Режим доступу до ресурсу: <https://forum.unity.com/threads/dots-is-awesome-but-what-will-happen-to-monobehaviours.761246/>.
8. Harabor, D., Grastien, A. A Systematic Review and Analysis of Intelligence-Based Pathfinding Algorithms [Електронний ресурс] – Режим доступу до ресурсу: <https://www.mdpi.com/2076-3417/12/11/5499>.

					ІАЛЦ.467200.003 ПЗ	Арк.
						78
Змн.	Арк.	№ докум.	Підпис	Дата		

9. Stunlock Studios V Rising: Behind the vampire realm built on 1,600 ECS systems [Електронний ресурс] – Режим доступу до ресурсу:

<https://unity.com/resources/stunlock-studios-v-rising>.

- 10.Unity Technologies EntityComponentSystemSamples Repository

[Електронний ресурс] – Режим доступу до ресурсу:

<https://github.com/Unity-Technologies/EntityComponentSystemSamples>.

					ІАЛЦ.467200.003 ПЗ	Арк.
						79
Змн.	Арк.	№ докум.	Підпис	Дата		

ДОДАТОК 1

Система порівняльного тестування ефективності
підходів DOTS та OOP в середовищі Unity

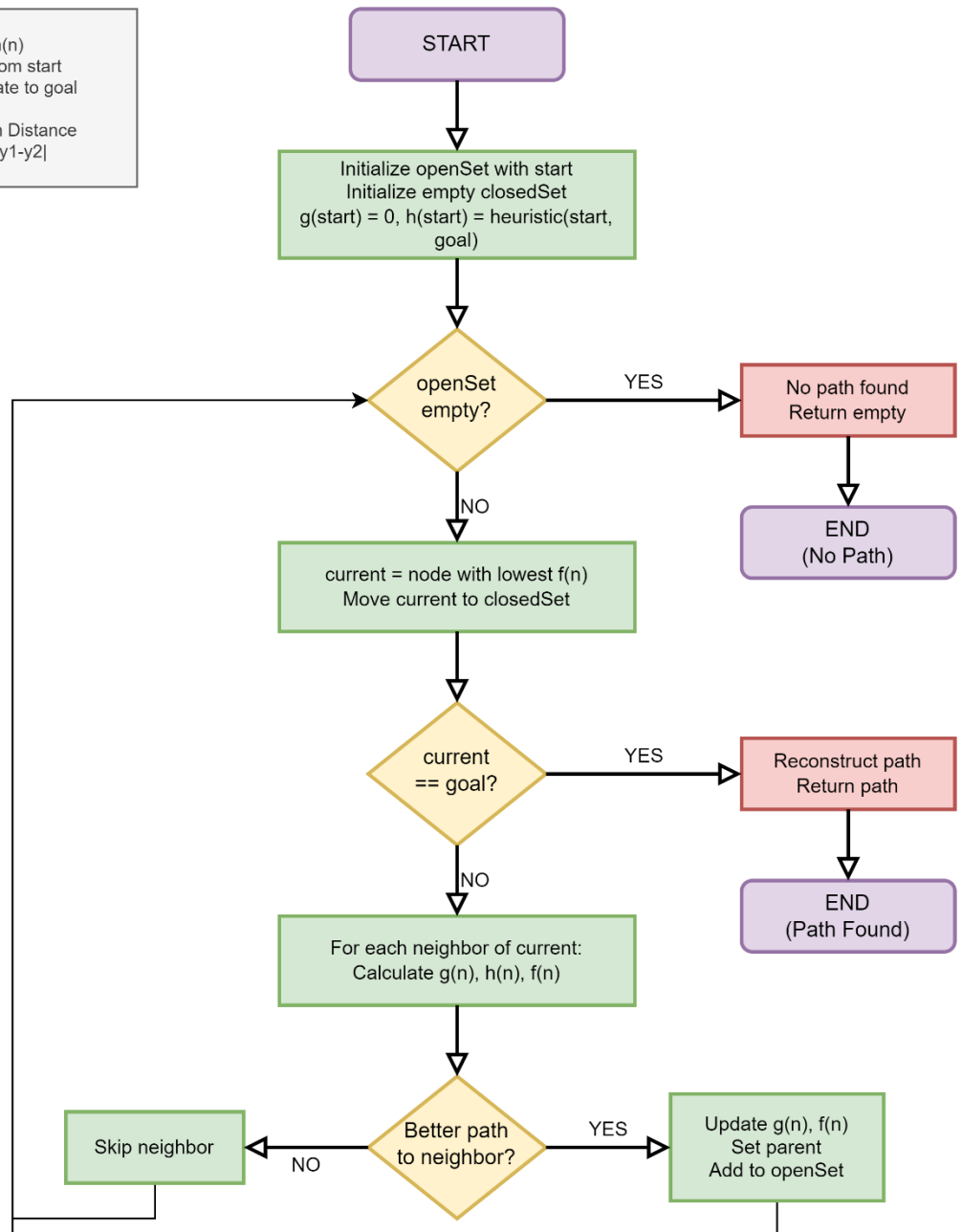
Діаграма імплементації A* алгоритму пошуку

ІАЛЦ.467200.004 Д1

Аркушів 1

Київ 2025 р

$f(n) = g(n) + h(n)$
 $g(n)$: actual cost from start
 $h(n)$: heuristic estimate to goal
 Heuristic: Manhattan Distance
 $h(n) = |x1-x2| + |y1-y2|$



					ІАЛЦ.467200.004 Д1		
Змн.	Арк.	№ докум.	Підпис	Дата	Система порівняльного тестування ефективності підходів DOTS та ООР в середовищі Unity Додаток 1		
Розроб.		Олійник В.В.					
Перевір.		Череватенко О.В.					
Н. Контр.		Пономаренко А.М.					
Затверд.							
					Лім.	Арк.	Акрушів
						1	1
					КПІ ім. Ігоря Сікорського, ФІОТ, ІМ-11		

ДОДАТОК 2

Система порівняльного тестування ефективності
підходів DOTS та OOP в середовищі Unity

Діаграма взаємодії компонентів між DOTS та
MonoBehaviour реалізаціями

ІАЛЦ.467200.005 Д2

Аркушів 1

Київ 2025 р

ДОДАТОК 4

Система порівняльного тестування ефективності підходів
DOTS та OOP в середовищі Unity

Текст програмного коду
ІАЛЦ.467200.006 ДЗ

Аркушів 7

Київ 2025 р

1. GridManager.cs - Менеджер сітки

```
public class GridManager : MonoBehaviour
{
    // Розмір сітки для навігації
    public int2 gridSize = new int2(20, 20);

    // Відсоток перешкод на сітці (0-50%)
    [Range(0f, 0.5f)]
    public float obstaclePercentage = 0.2f;

    // Двовимірний масив для зберігання інформації про прохідність клітинок
    private bool[,] walkableGrid;

    // Версія сітки для відстеження змін
    private int gridVersion = 0;

    void InitializeGrid()
    {
        // Створюємо нову сітку з заданими розмірами
        walkableGrid = new bool[gridSize.x, gridSize.y];

        // Спочатку всі клітинки прохідні
        for (int x = 0; x < gridSize.x; x++)
        {
            for (int y = 0; y < gridSize.y; y++)
            {
                walkableGrid[x, y] = true;
            }
        }

        // Генеруємо випадкові перешкоди
        if (generateRandomObstacles)
        {
            GenerateRandomObstacles();
        }

        // Збільшуємо версію сітки для інформування систем про зміни
        gridVersion++;
    }

    // Перевіряє чи можна пройти через клітинку
    public bool IsWalkable(int2 gridPos)
    {
        // Перевіряємо межі сітки
        if (gridPos.x < 0 || gridPos.x >= gridSize.x ||
            gridPos.y < 0 || gridPos.y >= gridSize.y)
            return false;

        return walkableGrid[gridPos.x, gridPos.y];
    }
}
```

					ІАЛЦ.467200.006 ДЗ			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Олійник В.В.			Система порівняльного тестування ефективності підходів DOTS та ООР в середовищі Unity Додаток 3	Лім.	Арк.	Акрушів
Перевір.		Череватенко О.В.					1	7
						КПІ ім. Ігоря Сікорського, ФІОТ, ІМ-11		
Н. Контр.		Пономаренко А.М.						
Затверд.								

2. PathfindingRequest.cs - Запит на пошук шляху

```
// DOTS версія - компонент для ECS
public struct PathfindingRequest : IComponentData
{
    public int2 startPosition;    // Початкова позиція
    public int2 targetPosition;   // Цільова позиція
    public bool isProcessing;     // Чи обробляється зараз
    public bool hasResult;        // Чи є результат
}

// Буфер для зберігання результату шляху
public struct PathResult : IBufferElementData
{
    public int2 position;    // Позиція точки шляху
}

// MonoBehaviour версія - клас для традиційного підходу
public class PathfindingRequest
{
    public int id;                                // Унікальний ідентифікатор
    public int2 startPosition;                    // Початкова позиція
    public int2 targetPosition;                   // Цільова позиція
    public List<int2> pathPositions;              // Знайдений шлях
    public bool success;                          // Чи успішно знайдено шлях
    public float processingTime;                  // Час обробки в мілісекундах
    public bool wasTruncated;                     // Чи був шлях обрізаний через
ліміти
}
```

3. Pathfinder.cs - Алгоритм пошуку (DOTS)

```
[BurstCompile]
public struct ParallelPathfindingJob : IJobParallelFor
{
    [ReadOnly] public NativeArray<bool> walkableGrid;    // Сітка прохідності
    [ReadOnly] public int2 gridSize;                    // Розміри сітки
    [ReadOnly] public NativeArray<PathfindingJobData> requests; // Запити на
пошук
    [WriteOnly] public NativeArray<PathfindingJobResult> results; // Результати

    // Основний метод пошуку шляху для кожного запиту
    public void Execute(int index)
    {
        var request = requests[index];
        if (!request.shouldProcess)
        {
            results[index] = new PathfindingJobResult(0);
            return;
        }

        // Викликаємо алгоритм A* для пошуку шляху
        var path = FindPath(request.startPosition, request.targetPosition);
        results[index] = path;
    }

    // Реалізація алгоритму A* (A-зірка)
    private PathfindingJobResult FindPath(int2 start, int2 target)
    {
        // Перевіряємо валідність початкової та кінцевої точок
        if (!IsValidAndWalkable(start) || !IsValidAndWalkable(target))
            return new PathfindingJobResult(0);
    }
}
```

					ІАЛЦ.467200.006 ДЗ	Арк.
						2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

// Список відкритих вузлів для розгляду
var openSet = new NativeList<PathNode>(256, Allocator.Temp);

// Множина закритих вузлів (вже розглянутих)
var closedSet = new NativeHashSet<int>(gridSize.x * gridSize.y,
Allocator.Temp);

// Мапа для відстеження батьківських вузлів (для відновлення шляху)
var cameFrom = new NativeHashMap<int, int>(gridSize.x * gridSize.y,
Allocator.Temp);

// Створюємо початковий вузол
var startNode = new PathNode
{
    position = start,
    gCost = 0, // Вартість від початку
    hCost = CalculateDistance(start, target), // Евристична вартість
    index = GetIndex(start)
};
startNode.fCost = startNode.gCost + startNode.hCost; // Загальна
вартість

openSet.Add(startNode);

// Основний цикл алгоритму A*
while (openSet.Length > 0)
{
    // Знаходимо вузол з найменшою загальною вартістю
    int currentNodeIndex = GetLowestFCostIndex(openSet);
    var currentNode = openSet[currentNodeIndex];

    // Якщо досягли цілі - відновлюємо шлях
    if (currentNode.position.Equals(target))
    {
        result = ReconstructPath(cameFrom, currentNode.index, start,
target);
        break;
    }

    // Переміщуємо вузол з відкритого в закритий список
    openSet.RemoveAtSwapBack(currentNodeIndex);
    closedSet.Add(currentNode.index);

    // Перевіряємо всіх 8 сусідів (включаючи діагональні)
    for (int dx = -1; dx <= 1; dx++)
    {
        for (int dy = -1; dy <= 1; dy++)
        {
            if (dx == 0 && dy == 0) continue; // Пропускаємо поточну
клітинку

            int2 neighborPos = currentNode.position + new int2(dx, dy);

            // Перевіряємо валідність та прохідність сусіда
            if (!IsValidAndWalkable(neighborPos)) continue;

            int neighborIndex = GetIndex(neighborPos);
            if (closedSet.Contains(neighborIndex)) continue; // Вже
розглянутий

            // Вартість руху (10 для прямих, 14 для діагональних)
            int moveCost = (dx == 0 || dy == 0) ? 10 : 14;

```

					ІАЛЦ.467200.006 ДЗ	Арк.
						3
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        int tentativeGCost = currentNode.gCost + moveCost;

        // Логіка оновлення або додавання вузла до відкритого
        списку
        // ...
    }
}

return result;
}

// Евристична функція для оцінки відстані (Manhattan + діагональ)
private int CalculateDistance(int2 a, int2 b)
{
    int dx = math.abs(a.x - b.x);
    int dy = math.abs(a.y - b.y);
    // Повертає точну вартість руху з урахуванням діагоналей
    return dx > dy ? 14 * dy + 10 * (dx - dy) : 14 * dx + 10 * (dy - dx);
}
}

```

4. PathfindingSystem.cs - Система управління (DOTS)

```

public partial struct PathfindingSystem : ISystem
{
    // Запити для різних типів сутностей
    private EntityQuery pendingQuery;    // Сутності що чекають обробки
    private EntityQuery processingQuery; // Сутності що обробляються

    // Кешована інформація про сітку
    private NativeArray<bool> walkableGrid;
    private int2 gridSize;
    private int cachedGridVersion;

    public void OnUpdate(ref SystemState state)
    {
        // Завершуємо попередні завдання та обробляємо результати
        if (hasActiveJob)
        {
            if (previousJobHandle.IsCompleted)
            {
                previousJobHandle.Complete();
                ProcessCompletedJobs(ref state); // Записуємо результати назад
                до сутностей
                hasActiveJob = false;
            }
            else
            {
                return; // Чекаємо завершення попередніх завдань
            }
        }

        // Оновлюємо дані сітки при необхідності
        UpdateGridDataIfNeeded();
        if (!walkableGrid.IsCreated) return;

        // Запускаємо нові завдання пошуку шляху
        StartNewPathfindingJobs(ref state);
    }

    private void StartNewPathfindingJobs(ref SystemState state)
    {

```

					ІАЛЦ.467200.006 ДЗ	Арк.
						4
Змн.	Арк.	№ докум.	Підпис	Дата		

```

// Фільтруємо сутності що потребують обробки
var filteredEntities = new NativeList<Entity>(Allocator.TempJob);

foreach (var (request, entity) in
SystemAPI.Query<RefRO<PathfindingRequest>>()
    .WithEntityAccess().WithNone<PathfindingProcessing>())
{
    // Додаємо тільки ті що не мають результату і не обробляються
    if (!request.ValueRO.hasResult && !request.ValueRO.isProcessing)
    {
        filteredEntities.Add(entity);
    }
}

if (filteredEntities.Length == 0) return;

// Підготовка даних для паралельного завдання
var jobRequests = new NativeArray<PathfindingJobData>(entities.Length,
Allocator.TempJob);
var jobResults = new NativeArray<PathfindingJobResult>(entities.Length,
Allocator.TempJob);

// Створення та запуск паралельного завдання
var pathfindingJob = new ParallelPathfindingJob
{
    walkableGrid = walkableGrid,
    gridSize = gridSize,
    requests = jobRequests,
    results = jobResults
};

// Планування виконання з розпаралелюванням
var pathfindingHandle = pathfindingJob.Schedule(entities.Length, 1,
state.Dependency);

state.Dependency = pathfindingHandle;
previousJobHandle = pathfindingHandle;
hasActiveJob = true;
}
}

```

5. PathfindingSystem.cs - Система управління

(MonoBehaviour)

```

public class PathfindingSystem : MonoBehaviour
{
    // Списки для управління запитами
    private List<PathfindingRequest> pendingRequests = new
List<PathfindingRequest>();
    private List<PathfindingRequest> completedRequests = new
List<PathfindingRequest>();

    // Створення нового запиту на пошук шляху
    public int RequestPath(int2 start, int2 target)
    {
        var request = new PathfindingRequest
        {
            id = nextRequestId++,
            startPosition = start,
            targetPosition = target,
            isProcessing = false,
            hasResult = false,

```

					ІАЛЦ.467200.006 ДЗ	Арк.
						5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        requestTime = DateTime.Now
    };

    pendingRequests.Add(request);
    return request.id;
}

void Update()
{
    // Обробляємо всі очікуючі запити негайно (без обмежень)
    ProcessAllPendingRequests();
}

private void ProcessAllPendingRequests()
{
    // Обробляємо ВСІ запити відразу - без throttling
    while (pendingRequests.Count > 0)
    {
        var request = pendingRequests[0];
        pendingRequests.RemoveAt(0);

        ProcessPathfindingRequestImmediate(request);
    }
}

private void ProcessPathfindingRequestImmediate(PathfindingRequest request)
{
    // Вимірюємо час виконання
    var startTime = System.Diagnostics.Stopwatch.StartNew();

    request.isProcessing = true;

    // Викликаємо алгоритм пошуку шляху
    var path = FindPath(request.startPosition, request.targetPosition);

    // Заповнюємо результати
    request.pathPositions = path;
    request.success = path.Count > 0 && path[path.Count -
1].Equals(request.targetPosition);
    request.wasTruncated = path.Count >= maxPathLength;
    request.processingTime = (float)startTime.Elapsed.TotalMilliseconds;
    request.hasResult = true;
    request.isProcessing = false;

    completedRequests.Add(request);
}
}

```

6. DataCollector.cs - Збір та аналіз даних

```

public class DataCollector : MonoBehaviour
{
    // Структури для зберігання статистики
    private PathfindingData currentData;           // Поточні дані
    private List<float> throughputHistory;         // Історія пропускної
здатності
    private float currentPathsPerSecond;          // Поточна швидкість обробки

    void Update()
    {
        CollectCompletedPaths();                  // Збираємо завершені шляхи
        UpdateThroughputMetrics();                 // Оновлюємо метрики продуктивності
    }
}

```

					ІАЛЦ.467200.006 ДЗ	Арк.
						6
Змн.	Арк.	№ докум.	Підпис	Дата		

```

void CollectCompletedPaths()
{
    // DOTS версія: збираємо дані з Entity Manager
    var query = entityManager.CreateEntityQuery(typeof(PathfindingRequest),
    typeof(PathResult));
    var entities = query.ToEntityArray(Allocator.TempJob);

    foreach (var entity in entities)
    {
        if (processedEntities.Contains(entity)) continue;

        // Перевіряємо чи є результат
        var request =
entityManager.GetComponentData<PathfindingRequest>(entity);
        if (!request.hasResult || request.isProcessing) continue;

        // Отримуємо шлях з буфера
        var pathBuffer = entityManager.GetBuffer<PathResult>(entity);

        // Аналізуємо результат та зберігаємо статистику
        var record = new PathRecord
        {
            id = nextId++,
            start = request.startPosition,
            end = request.targetPosition,
            success = /* логіка перевірки успішності */,
            length = pathBuffer.Length,
            timeMs = 0f // В DOTS немає індивідуального таймінгу
        };

        currentData.paths.Add(record);
        processedEntities.Add(entity);
    }

    UpdateStatistics(); // Оновлюємо загальну статистику
}

// Розрахунок метрик пропускної здатності
void UpdateThroughputMetrics()
{
    float currentTime = Time.realtimeSinceStartup;
    float timeSinceLastUpdate = currentTime - lastThroughputUpdate;

    if (timeSinceLastUpdate >= throughputUpdateInterval)
    {
        int currentPathCount = currentData.paths.Count;
        int pathsCompleted = currentPathCount - pathsAtLastUpdate;

        // Розраховуємо поточну швидкість (шляхів/секунду)
        currentPathsPerSecond = pathsCompleted / timeSinceLastUpdate;
        throughputHistory.Add(currentPathsPerSecond);

        // Зберігаємо тільки останні 60 зразків (1 хвилина)
        if (throughputHistory.Count > 60)
        {
            throughputHistory.RemoveAt(0);
        }

        lastThroughputUpdate = currentTime;
        pathsAtLastUpdate = currentPathCount;
    }
}

```

					ІАЛЦ.467200.006 ДЗ	Арк.
						7
Змн.	Арк.	№ докум.	Підпис	Дата		